Writfold implementation review

Citation proofs: pages 2-7. Run Watch: 57-63. Incident pilot: 64-66. General discovery and Metamath: 67-68.



First / Prove citation binding

Next / Qualify the compact reader

Then / Enable proven proposal paths

Later / Evaluate the new views

What opens proposal admission

Prove the rule in Lean, then kernel-check its application to the exact proposal, witness and predecessor. Required production, protocol and authority checks must also pass.

Release inventory: 27 stages, 34 gates, 23 Verus obligations, 16 protocol models and 164 scenarios. E01-E12 and CIT01-CIT08 are proposed Lean targets. New proof receipts are pending.

What the implementing agent should do first

Resume verified work from its actual commit. The full release inventory is available when a dependency calls for it.

1 NOW / P03C	Reconcile the existing proof work. Define citation binding and admission, then prove CIT01-CIT08. Keep proposals in candidate state while proofs are missing.
2 NEXT / P05C	Verify the production Rust checker and build the compact reader. Reproduce the copied-line failure and compare token cost with targeted shell reads.
3 THEN / P07 onward	Enable each qualified kind after its Lean rule and exact instance certificate pass. Recheck authority and protocol evidence at admission.
4 LATER / experiments	Add acquisition hints and focused Obsidian explanations. Try a read-only run monitor after ledger evidence is available.
5 PARKED	Reconsider heterogeneous TTLs, a standalone graph browser and automated learning rules only when a measured use case justifies them.

Daily handoff: one active obligation, its owner, the missing evidence and the next reproducible command. See PRIORITIES.md.

Every accepted proposal has a proof-backed rule

Drafting, experiments and review can proceed before admission. The authoritative transition has a separate gate.

Trusted profile

The released engine selects the required Lean theorems, proof bundle and qualified implementation for this proposal kind.

Concrete proposal

The Lean worker checks this candidate and witness at the exact predecessor. Its authenticated instance receipt names the evidence and intended use.

Live conditions

Check policy, grants, freshness, gate receipts and independent review where required.

Blocked

Missing or stale proof, unsupported kind or invalid witness. Keep selected state unchanged.

Admission decision

Require the rule bundle and instance receipt. Recheck current premises at the authoritative boundary.

Conditional publication

A successful decision permits the applicable design transition or release CAS. Retain the receipt.

A new semantic rule must qualify under the old trusted floor and independently prove its successor. The candidate cannot select its own proof requirements.

Four compact operations over retained evidence

Agents exchange short scoped handles. The engine retains the complete source, selector and provenance binding.

find	Discover candidates at a pinned release and named context. Show search coverage, roles and omissions alongside each scoped handle.
peek	Expand an authorized handle to the exact retained excerpt. Display bounds, provenance and required caveats within the budget.
cite	Expand the canonical public citation and prepare the proposed binding. Admission still requires the qualified rule and witness.
compare	Compare an explicit target revision. Report justified correspondence, multiple candidates, proved deletion/replacement or unknown.

Coordinates belong to a representation

A byte interval in a Git blob and a UTF-16 range in a Google Docs tab use different types. A quote hash helps discovery. Identical text can appear twice.

The copied line is the wrong successor

Reproduced with the actual kctx constructor and current-location classifier in the archived 1c17b94 source.

Original revision

```
1  def intended():
2      return DEFAULT
```

New revision

```
1  def other():
2      return DEFAULT
3
4  def intended():
5      return DEFAULT
```

Observed legacy advisory

The normalized text matches lines 2 and 5. The helper reports moved at line 2, inside other(). The historical source remains intact.

Required revision-6 result

Expose both candidates. Selecting the intended successor requires additional correspondence evidence. Preserve the historical binding.

Run tools/reproduce_duplicate_citation.py. This reproduces a legacy observation. CIT03 and CIT04 still require checked Lean proofs.

Three different clocks for the same asset

Choose a policy for the intended decision. A source medium alone says little about how quickly its claims can become stale.

Recheck scheduling

The catalog names the detector, owner, last successful source check, and polling or event hint. These help choose acquisition work.

Evidence freshness

The signed context policy assesses the original observation dependencies at an explicit time. An ordinary reread preserves their ages.

Retention

Release, citation, pending-operation and reader roots govern which bytes remain available. Expiry alone cannot destroy history.

Historical design	A retained source shows what was written at that revision. Its applicability to today's rollout is a separate judgment.
Current deployment	A document can stay byte-identical while the deployed version changes. Obtain evidence for the actual target and intended use.

First use existing named contexts. Different TTLs inside a single witness would change semantics and require a new proved policy extension.

What would justify the next feature?

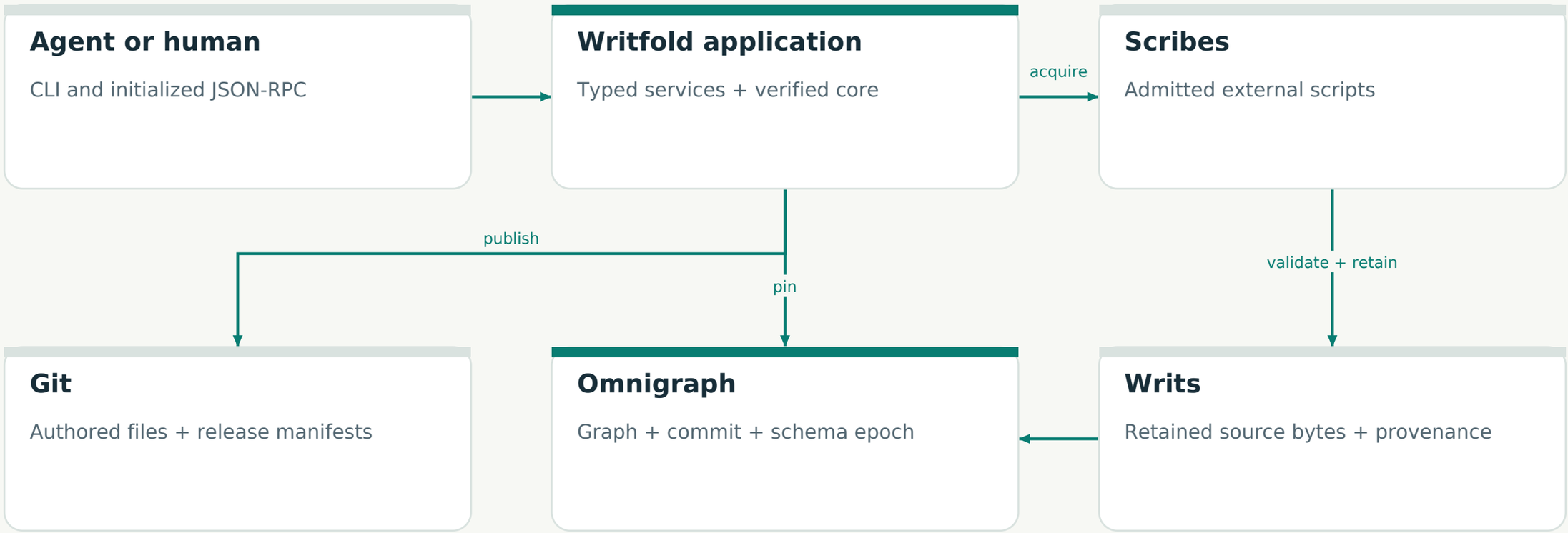
Record small, falsifiable decisions before expanding the release. Earlier proof gates continue to govern proposal admission.

Compact citations	Compare repeated inspect-and-cite tasks with kctx and targeted rg. Count real input/output tokens, retries, latency, wrong attachments and omissions.
The starting measurement	One fixture returned 4,620 whole-file bytes, 199 targeted rg bytes and 361 bytes of illustrative JSON. The first lookup alone favors rg.
Focused Obsidian views	Ask whether Tarek can trace a blocker to its test and citation. Show one scoped diagram with explicit edge meanings and an ordinary table.
Read-only Ratatui pilot	Pages 57-63 show the approved run views. Compare blocker, revision and next-evidence lookup against plain CLI output on the same run.
Decision	Promote a successful experiment through the normal proof and review path. Keep an inconclusive result provisional and record its evidence.

The 27-stage release plan provides traceability. The immediate working queue has one mathematical foundation and one checked reader.

Small core. Explicit boundaries.

The catalog routes capabilities; external scribes acquire evidence without authority to publish claims.



The completed Git release pins the exact Omnigraph graph/commit/epoch pin, signed operation and retained evidence closure. Prepared objects and recovery journals are not publication authority.

Runtime: Linux/WSL x86_64. Native Rust application + managed Omnigraph + explicit Git dependency. Proof tools and Python remain outside the core runtime.

Prove the contract you actually ship.

Keep mathematical semantics, executable verification, protocol safety and real-process evidence separate.

LEAN

Language and judgments

353 baseline declarations in T01-T22. Engineering E01-E12 and citation CIT01-CIT08 carry separate pending proof obligations.

VERUS

Actual executable decisions

V01-V23 cover knowledge, engineering, citations and admission. Production callers establish preconditions.

TLA+ / TLC

Interleavings and failures

M01-M16; knowledge, ledger, effects, skills, learning and projection models. Include environment changes and calibrated negative cases.

TLAPS

Inductive safety proofs

Init, every action, frame/stutter cases and cross-model bridges. Temporal claims stay within executed tool support.

Trusted boundaries remain visible: source truth, cryptographic libraries, OS durability, Git/Omnigraph semantics, sandbox enforcement and cross-prover correspondence. Tests support those boundaries; they do not turn them into proofs.

More contexts. The same inference boundaries.

Portfolios and design binding add explicit authority without changing the core inference rules.

PORTFOLIOS

One release, separate assessments

A complete signed family of named contexts. Each new claim has an exact context and witness assignment. No source, receipt or policy borrowing across contexts.

DESIGN BINDING

Approve intent, then bind implementation

Accept a stable design before code exists. Bind an exact source and target later. Rebinding preserves acceptance, but old-target passes stay historical.

Historical authority is not upgraded by relabelling.

Completed schema-2 releases keep their original bytes and signature domain. Pending old writes cannot resume through the schema-3 writer. Queries name a context; forward selection preserves current corrections and policy.

New stages: P03P portfolios and P08D design binding. New proofs: V12, V13 and M09. Full contract: plan/11.

Use the patterns. Keep the concrete engine.

Pinned to private tknawara/playbook at e7d991e; the new view channel follows I1/I2/I3 without a generic framework.

S1 / S2

Explicit decisions and checked stages

Production Rust predicates; exact prepared plans; callers establish preconditions.

S3 / I3

Controlled effects and recovery

One owner per field; admitted capabilities; durable closure before visibility.

I1

Prepared presentation

CLI and RPC render the same immutable engine view. Rendering performs no IO or mutation.

I2

Stamped results and owned lifetimes

Check session/request/source/grant identity. Cancellation holds capacity until physical teardown.

WF21 keeps the adoption record, production mapping, diagrams and this companion current. Updating Writfold does not require editing the entire upstream playbook. Private reference material is not published on the website.

A minimal home for writfold.dev.

One useful page, inspired by Gripsack’s current design. Complete section copy and acceptance rules are in plan/10.

writfold

start scribes docs GitHub

Keep the context. Know what supports it.

Bring evidence from your tools into one durable knowledge base. Keep its source, history and approval attached.

get started

Linux x86_64 · WSL2

Keep the evidence.

Exact source and history.

Review what changes.

Explicit approval.

Carry context forward.

Support and visible gaps.

start with one source

One real workflow. Then connect a tool with a small scribe.

install / offline bundle docs assurance source

01 Hero

One clear promise. No fake install command.

02 Three capabilities

Evidence, reviewed changes, reusable context.

03 One workflow

Tested commands and a real synthetic result.

04 Scribes

Simple scripts; explicit evidence boundaries.

05 Install + docs

Exact release and honest prerequisites.

06 Footer

Source, changelog and confirmed license.

A folded W, built as a real vector.

Light, dark, monochrome and favicon SVGs are included. The mark contains no embedded raster, scripts or external resources.



Light surface



Dark surface



Favicon

The upper-right facet folds inward. Keep the silhouette consistent across themes; change only its defined colors. Use transparent SVG for UI/README and the tiled favicon for browser tabs.

Writfold is settled. Scribes are supervised evidence adapters; writs are retained evidence records. Historical kctx identifiers and signed bytes keep their original identity.

One knowledge model. More ways to read it.

Obsidian is the first human view channel. Future HTML can consume the same prepared document.

Pinned knowledge

Release + context + policy + source closure. Filter the authorized audience before rendering.

Prepared document

Typed headings, facts, evidence and notices. Assessment clock and coverage stay attached.

Pure renderers

Portable Markdown and Obsidian today. HTML later; no parsing Markdown back into authority.

Scribes acquire

Supervised tools return retained evidence. They cannot publish claims or write the view vault.

Publisher owns effects

Validate paths and output closure. Install an immutable generation; activate conditionally.

Obsidian reads

Bases, Dataview and the CLI navigate a snapshot. Editing a note is not knowledge admission.

New proof boundary: V14 must verify faithful projection; M10 specifies generation publication and process lifetime. The app and plugins remain explicit tested boundaries.

A page for a person, with evidence one click deeper.

Design preview using synthetic content. This is not a screenshot or a native Obsidian qualification result.

WRITFOLD

Operations

Overview

Assets and attention

Checkout

Inventory

Evidence

Decisions

My reading notes

DEMO SNAPSHOT / OPERATIONS / 2 NOV 2026

Checkout

Accepts a purchase request and asks Inventory to reserve the items. This design relationship is supported; runtime health is outside this export.

! Owner needs evidence

No current owner record is included. Missing evidence does not mean there is no owner.

What supports this page

STATEMENT	SUPPORT	SCOPE
Depends on Inventory	Contract evidence	Design relationship
Reservation example passed	Test receipt example	One synthetic fixture

› Evidence details · Nearby: Inventory · Add a personal note

A short explanation, an unhidden gap, a small support table and explicit scope. Long provenance folds away; the only warning never does.

Tables that help you find the next useful question.

Core Bases for everyday browsing; pinned Dataview for curated relational dashboards.

Assets · core Bases

Table and card layouts share the same generation/context filters. No unscoped whole-vault query.

Checkout	Needs evidence
Inventory	Supported

Synthetic snapshot · 2 services · no live health claim

Needs attention · Dataview

Fixed DQL templates select assessed labels. No generated JavaScript and no inference in the plugin.

Checkout	Owner unknown

Synthetic snapshot · capped at 25 rows

Pin the actual bytes

Selected candidate: Dataview 0.5.68. The newer tag has a mismatched manifest; retain both observations.

Install with intent

Exact vault and admitted plugin ID. CLI install has no version flag: verify assets before enabling.

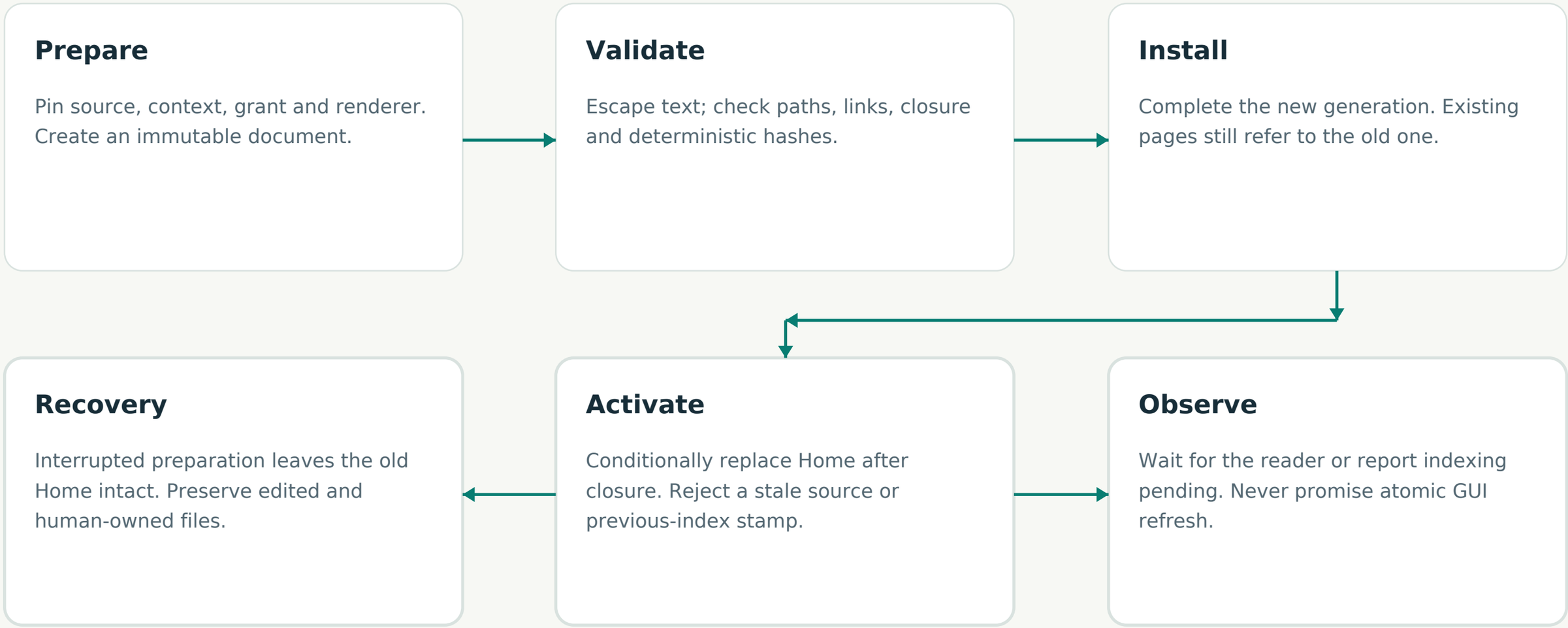
Keep it readable

Static notes and tables survive missing plugins. Outline, previews and backlinks help navigation.

WF23: real app/CLI versions, query results, indexing lag, light/dark/narrow reading and plugin-off fallback. These native checks remain to run on the PC.

New snapshot first. Navigation changes last.

Immutable generations preserve meaning when an old note remains open or the app indexes slowly.



M10 must prove modeled closure/stamp/ownership/lifetime safety. Native crash and watcher tests qualify filesystem/app assumptions. Cleanup is separate and respects retained references.

Omnigraph first. Qualify before integration.

One intended primary backend; an early blocking gate before Writfold relies on its storage contract.

Primary target

Omnigraph stores structured knowledge and typed relationships. Exact judgments and discovery have distinct contracts.

Legacy boundary

Dolt remains reference/import tooling and an explicit fallback decision. No second normal writer or automatic retry path.

Early gate: P01O

WF24 qualifies native semantics, epochs, restore, ownership and workload budgets on the actual WSL profile.

What the experiments actually established

Change an existing type	Old whole-node query refuses after a field addition.
Preserve V1; introduce V2	Old whole-node rows remain identical in the follow-up.
Branch from the older schema	Refused. Reading history is a different operation.
Production admission	Not established: the full foundation and integrated gates remain to run.

Keep the calculus and signed release boundary. Change the storage adapter, provisioned closure, migration protocol and proof assumptions explicitly.

Retained evidence: original comparison 12 assertions / 19 commands; follow-up 6 assertions / 7 commands. Neither is WSL/server/crash qualification.

A new schema gets a new epoch.

Old releases keep their graph, commit, schema and query contract. A migration prepares a separate successor.

Epoch 1: retained

Immutable physical schema. Old release pins its graph commit and versioned historical queries.

Approved mapping

Pin the source; account for every record. Preserve IDs, contexts, corrections and evidence.

Epoch 2: candidate

Initialize a separate graph. Reconstruct canonical state and verify the complete migration.

Old readers continue

Existing pins remain valid. Retain the old native roots, query bytes and compatible runtime profile.

Root, validate, CAS

Retain the complete candidate. Recheck source and predecessor; publish via the completed Git ref.

New readers select

Only the completed release selects epoch 2. Retire epoch 1 after every retention obligation ends.

No cross-schema branch shortcut. New proposals target the current release; genuine historical-schema replay uses an isolated verified restoration.

Required: V15 storage-pin/migration admission + M11 epoch safety, with bridges to publication, recovery, retention and views. These are proof obligations, not completed proofs.

Three semantic graphs. Two storage families.

One managed Omnigraph companion per trust realm. Each family retains independent schema epochs.

Context

Admitted claims, witnesses, sources, corrections and selected procedure definitions.

Execution

Procedure structure, actual runs, attempts, effects and checks. Definitions and instances stay distinct.

Learning

Observations, candidates, evaluation and promotion into new admitted revisions.

Corpus family

Admitted knowledge, skill/procedure revisions and accepted lessons. A completed Git release selects the exact native pin.

Activity family

Queryable run and learning projections from the durable EngineeringLedger. Exact watermarks; never effect authority.

Learning promotion crosses through the existing admission protocol: retain evidence, validate the candidate, then publish the completed Git release.

Two logical families can mean more than two physical graphs while historical epochs remain retained. No cross-graph transaction is assumed.

Which record can justify the next action?

Queryability, recorded history and current permission have different owners.

Corpus release	Exact graph/commit/epoch/query pin plus signed retained closure. Preserves version-4 knowledge authority.
EngineeringLedger	Immutable records and conditional stream heads select operational history. Reconstruct with the checked reducer.
Current guards	Applicable grants, ownership, subject bindings and external preconditions are checked at effect admission.
Activity graph	Derived projection with a watermark. A stale or forged status row cannot authorize a merge.
Recovery hints	Describe interrupted preparation. They never substitute for validated history, approval or observed outcomes.
Promotion evidence	Retained in corpus closure before publication, so activity pruning cannot erase a lesson's justification.

Local ledger origin relies on the qualified coordinator and private-filesystem boundary. Hash chains establish linkage; they do not authenticate arbitrary actors.

A realm has explicit ownership on disk.

Authoritative state stays on qualified Linux storage. Worktrees and the external BuildKit setup keep their chosen locations.

identity.json + releases.git/	Realm/KbId and local instance; retained Git release manifests and source objects.
objects/sha256/	Exact source bytes, package resources, ledger records, receipts and admission evidence.
ledger/heads/	Conditional stream-head records. Stable logical identity; exact immutable event prefix.
coordination/	Cross-run resource reservations and owner generations; stable managed lock names.
omnigraph/cluster/	Whole engine-owned root: corpus/activity epochs, native history and metadata.
deployment/ + bindings/	Pinned schemas/queries/profiles; separate local roots, endpoints and worktree locators.
Runtime, cache and views	Private process endpoints; disposable caches; separately owned Obsidian generations and Human notes.

Default realm root: $XDG_DATA_HOME/writfold/realms/<RealmId>/. Paths are locators, not portable identity. Native backup captures the complete qualified root.

Name the exact thing a check depends on.

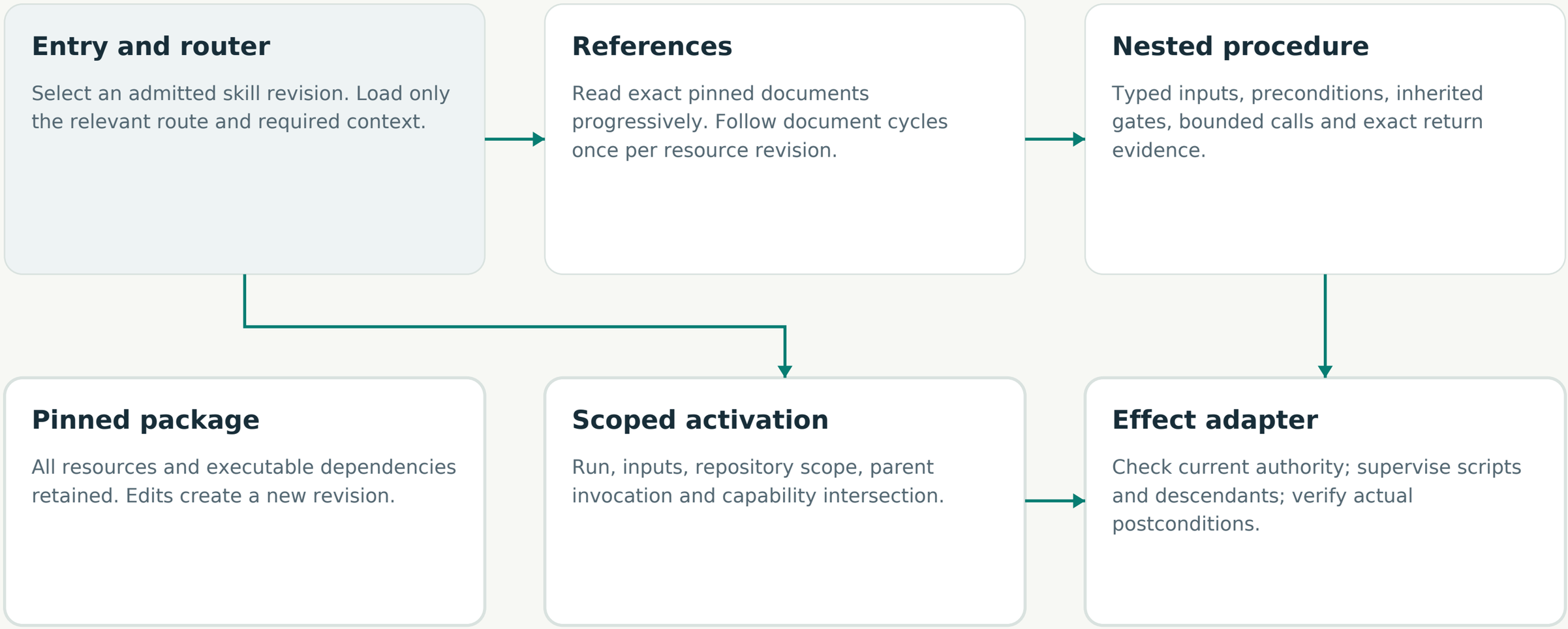
Logical IDs, native revisions, content digests and current authorization stamps remain distinct nominal types.

Package + dependency lock	Exact resource bytes and modes; flat resolved dependency graph. Document cycles do not create recursive hashes.
Skill + procedure revision	Entry/loader profile, package closure, typed control flow, contracts and required gates.
Repo vector + read set	Actual code/input snapshots across repos; exact corpus contexts, witnesses, coverage and assessment inputs.
Environment + attempt input	Tools, dependency locks, relevant configuration, fixtures and the exact step subject.
Effect intent + dispatch stamp	Stable logical operation survives recovery; current grant/head/owner/incarnation gets a fresh checked stamp.
Receipt + learning candidate	Exact result/evidence and proposed delta. Identity does not imply usefulness, approval or truth.
Engineering read cut	Corpus pin + activity pin + stream/inventory watermarks, visibility, time and completeness.

Proposed encoding: restricted deterministic CBOR + domain-separated SHA-256. Matching fingerprints still require freshness, authority, scope and complete evidence.

Keep the Markdown. Check the composition.

Existing Python and shell remain usable through admitted execution profiles; loading guidance does not grant effects.



Read, require, route, call and execute are different edges. A delivery record shows which text was supplied; it cannot prove model comprehension.

One task. An exact vector of repository revisions.

Illustrative task: cursor pagination across an API, a Rust SDK and a CLI while preserving existing consumers.

API service

Producer behavior and schema. Old consumer / new producer is an explicit compatibility case.

Rust SDK

Nominal cursor representation and decoder behavior. Changed bytes stale dependent receipts.

CLI

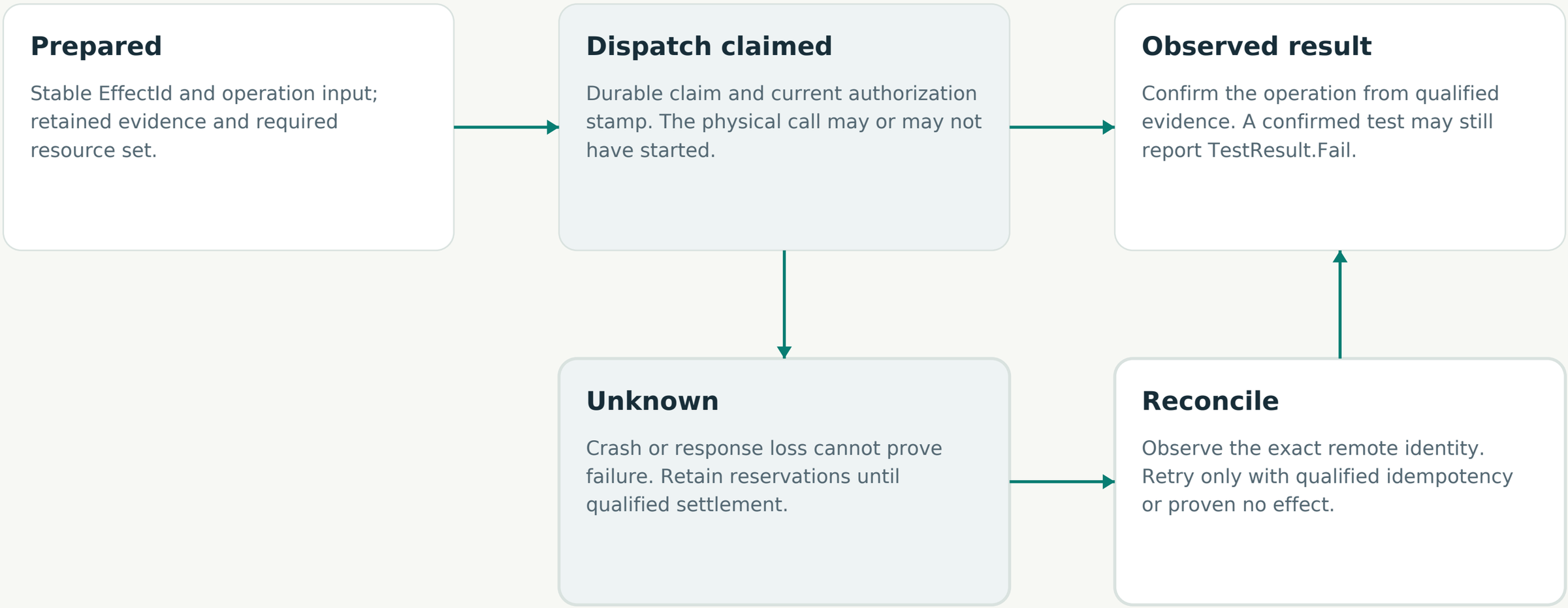
Passes opaque cursor values through. Dependency and release ordering stay explicit.

Research and challenge	Resolve assumptions and local conventions. Review alternatives and safe intermediate combinations.
Author and verify	Bind tests, proofs, review and documentation to actual targets, tools and fixtures.
Land and observe	Recheck upstream and merge candidates. Partial merges follow an admitted continuation or forward fix.

A green test for an older vector stays historical. Three green PRs are not an atomic multi-repo release. Uncontrolled external changes remain explicit model inputs.

A lost response is an unknown outcome.

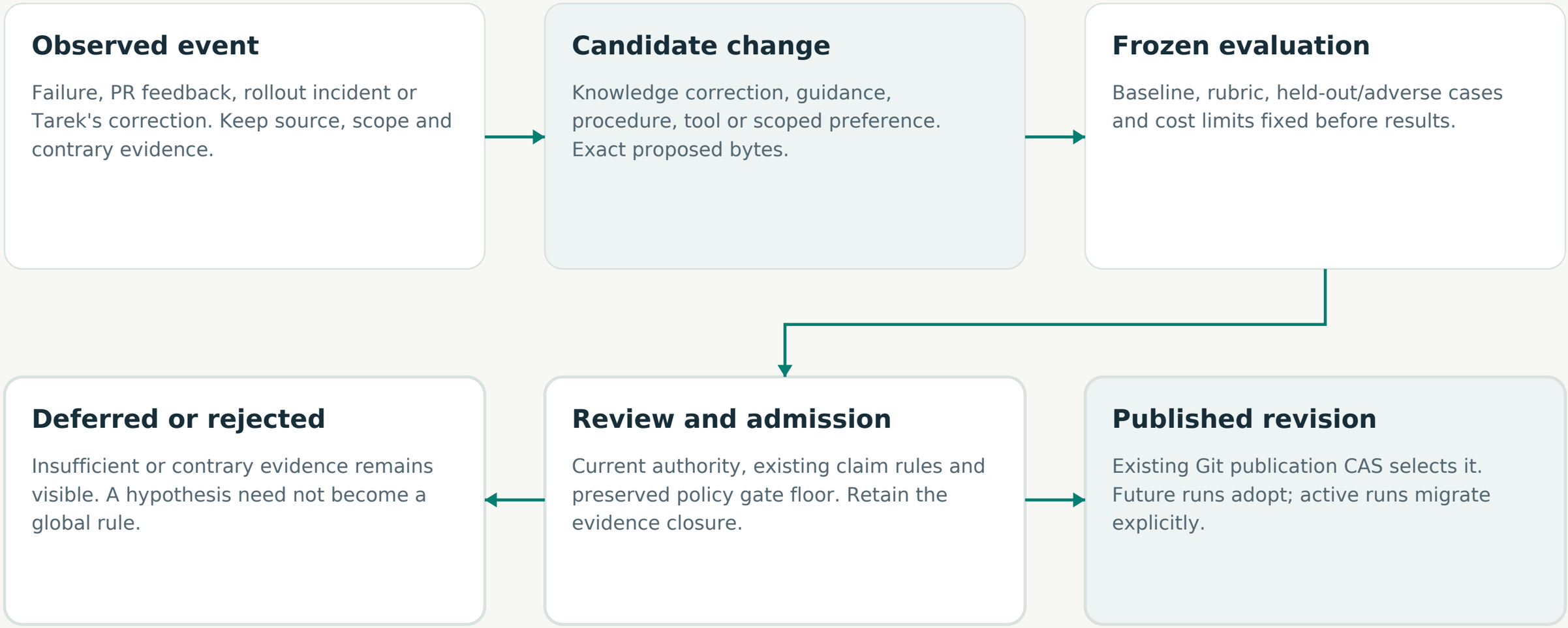
Persist a unique dispatch claim before external IO. Only its live owner may issue that claim once.



Even a crash after the claim but before the call recovers as Unknown until reconciled. There is no general exactly-once guarantee for external effects.

A useful lesson earns a new revision.

Capture observations throughout the run. Repair the current task while evaluating wider applicability separately.



Learning cannot modify its own governing evaluator, remove required gates or grant itself permission. Empirical usefulness and formal admission safety have separate evidence.

Three judgments, one explicit proof boundary.

Conservative extension of semantic 2.0.0. Existing T01-T22 remain unchanged; E01-E12 are new, unproved targets.

Usable knowledge

Which claim is supported and applicable in this exact named context?

Admissible action

Is this step enabled for the actual subject, current grant and owned resources?

Admissible improvement

Can this evaluated delta enter a new release without weakening the governing rules?

Lean E01-E12	Preservation, conservative extension, binding, capability/gate-floor constraints and conditional compatibility.
Verus V16-V21	Actual shipping decisions and safe caller conversions; no unverified easier twin.
TLA+ / TLC / TLAPS	M12-M16 plus bridges to existing models. Include time, revocation, upstream change and delayed external completion.

WF31 aggregates completed proofs and refinement at P11. Earlier stages contribute scoped receipts. No claim of infallible agents, universally useful lessons or verified arbitrary scripts.

A readable answer to: what is happening, and why?

Synthetic Obsidian design preview. The prepared document carries both corpus and activity read pins.

WRITFOLD

Change overview

Assumptions

Repositories

Checks and reviews

Effect recovery

Learning proposals

Evidence

Human notes

RUN DEMO-17 / CORPUS R42 / ACTIVITY THROUGH EVENT 38

Cursor pagination

Status: blocked on compatibility evidence

The old SDK rejects the proposed response field. The API change is being revised; the earlier green CLI check does not cover the new repository vector.

Assumption refuted

Old consumers tolerate added fields. Reproduction and decoder source retained.

Next permitted work

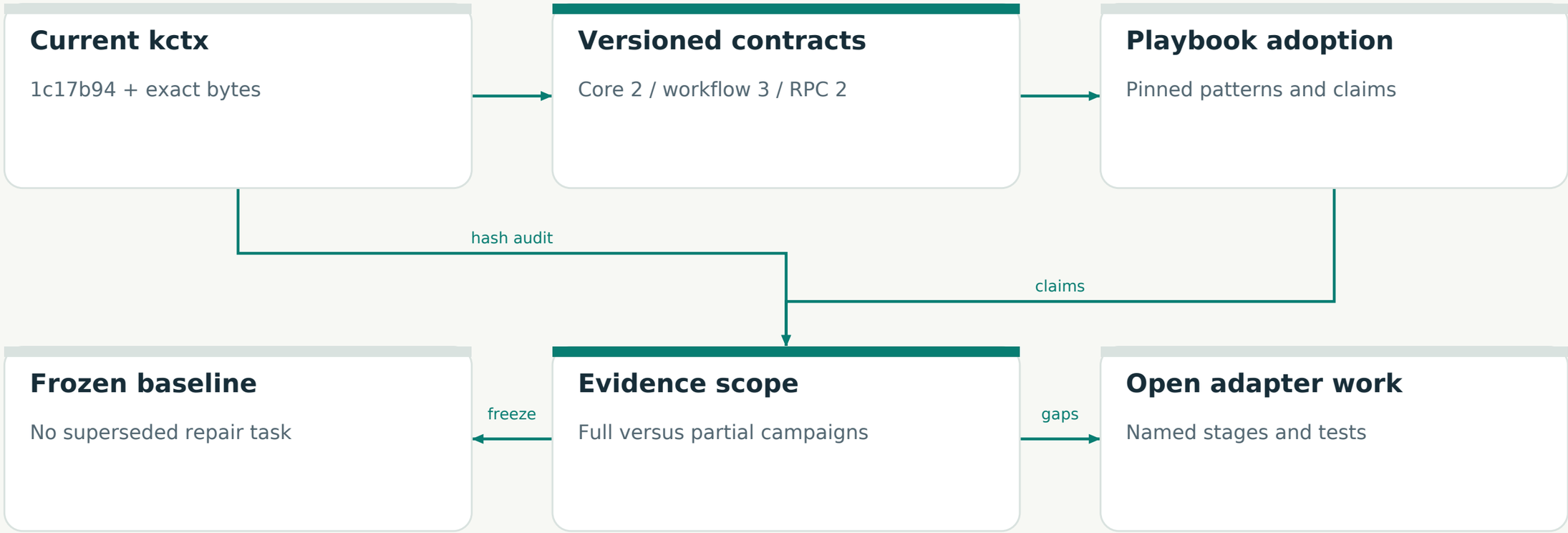
Revise compatibility strategy; rerun affected checks; update the linked design and PRs.

Learning proposal: test supported old consumers for response-schema extensions.

Projection lag and missing evidence remain visible. Editing this note does not admit knowledge or authorize an effect.

Freeze workflow 3 and adopt the playbook

Start from the current verified contracts and explicit remaining adapter work.



BUILD

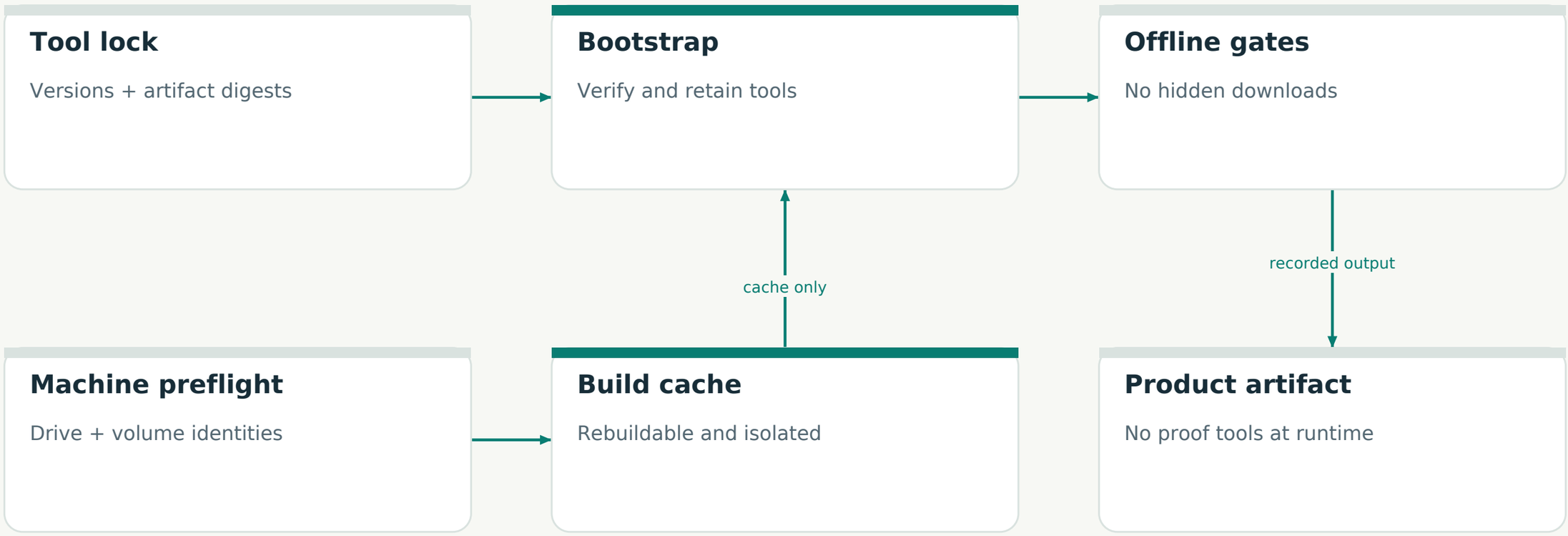
- Current-source baseline and version matrix
- Pinned playbook adoption + compatibility inventory
- Evidence applicability and unresolved-work ledger

EXIT GATE

- Current source matches the claimed report inputs; unrun/full-scope claims stay explicit.
- Completed fixes are preserved and unresolved behaviors have destination gates.
- Workflow 3, protocol 2 and private playbook adoption are explicit.

Pin the build and storage setup

Build the same admitted toolchain on WSL without filling the wrong disk.



BUILD

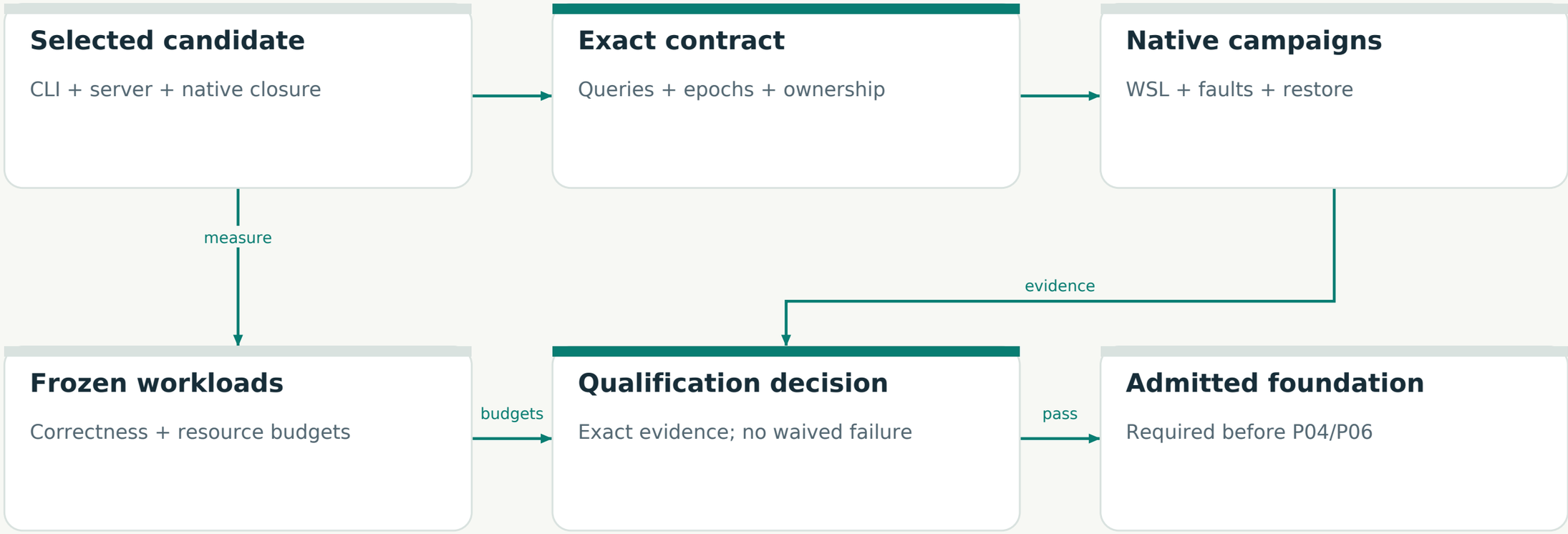
- Dockerfiles/compose.yaml + toolchains.lock.json
- scripts/gate + receipt schema + preflight
- Verus-to-musl feasibility artifact and offline tool closure

EXIT GATE

- Clean container build and offline rerun succeed from admitted bytes.
- Missing external mount fails before creating large output on the system disk.
- Verified executable body is linked into the smoke artifact with recorded target/features.

Qualify the Omnigraph foundation

Admit the chosen backend contract before storage-dependent protocols and integration rely on it.



BUILD

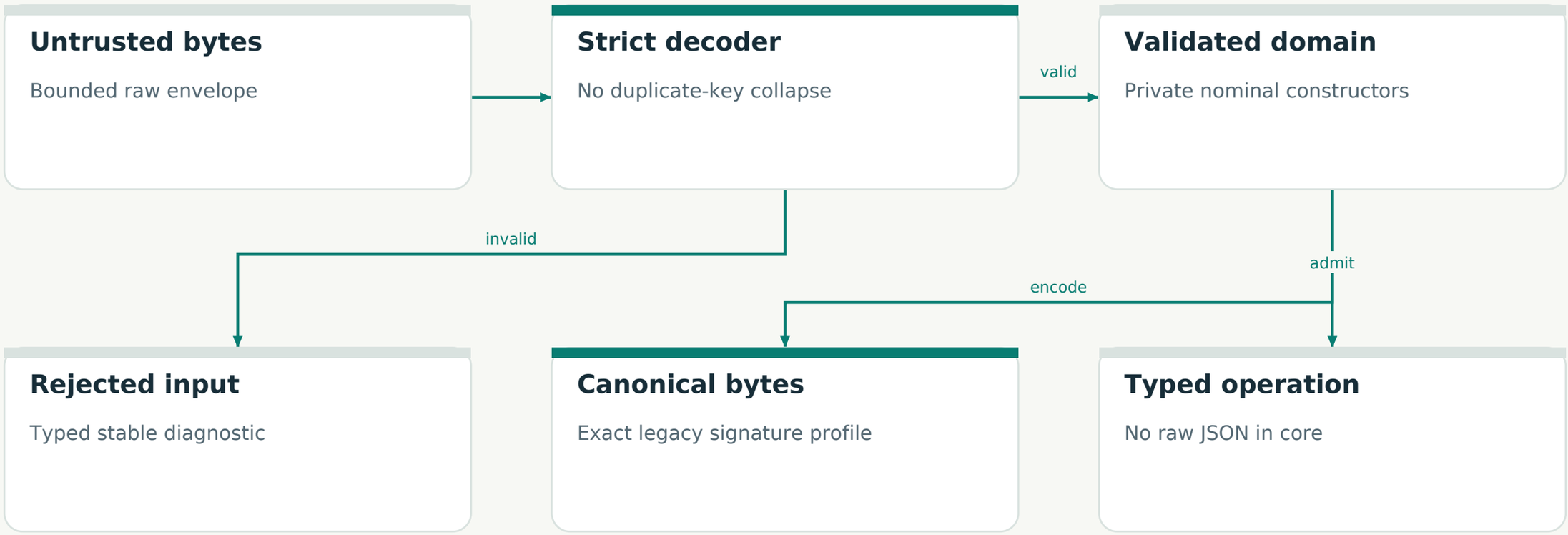
- Pinned native backend and compatibility contract
- WSL storage/history/failure/restore evidence
- Frozen budgets and explicit go/no-go receipt

EXIT GATE

- Full declared WF24 scope passes; smoke probes alone are insufficient.
- Old epoch reads and independent restore preserve exact identities.
- No second normal database writer or hidden fallback is introduced.

Nominal domain and strict wire

Make invalid identities, units and boundary data impossible to use silently.



BUILD

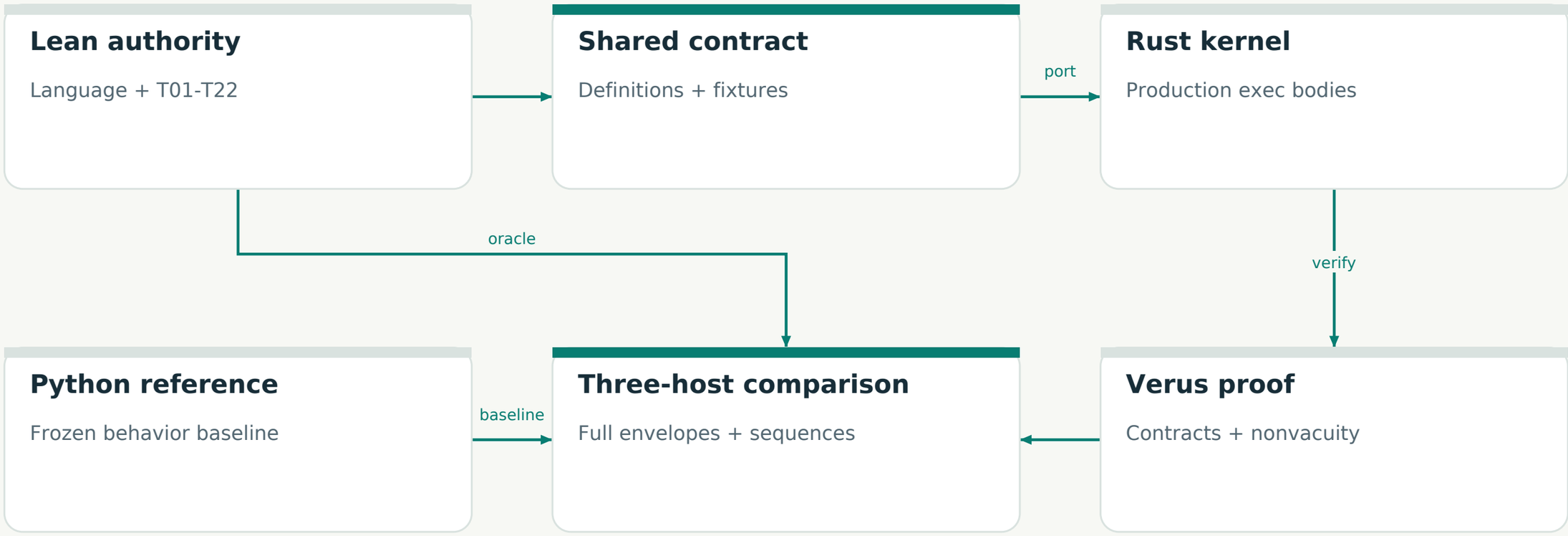
- Validated domain types + wire schemas/codecs
- Canonical byte/signature fixtures
- Architecture/size/type gate with deliberate failing controls

EXIT GATE

- Wrong namespace/unit/state fails compilation for the intended reason.
- Malformed, oversized, ambiguous and unsupported inputs fail before domain construction.
- Legacy signatures verify over unchanged exact bytes; no JSON-number narrowing.

Verify the executable semantic core

Port the actual calculus and verify Rust decision bodies without weakening semantics.



BUILD

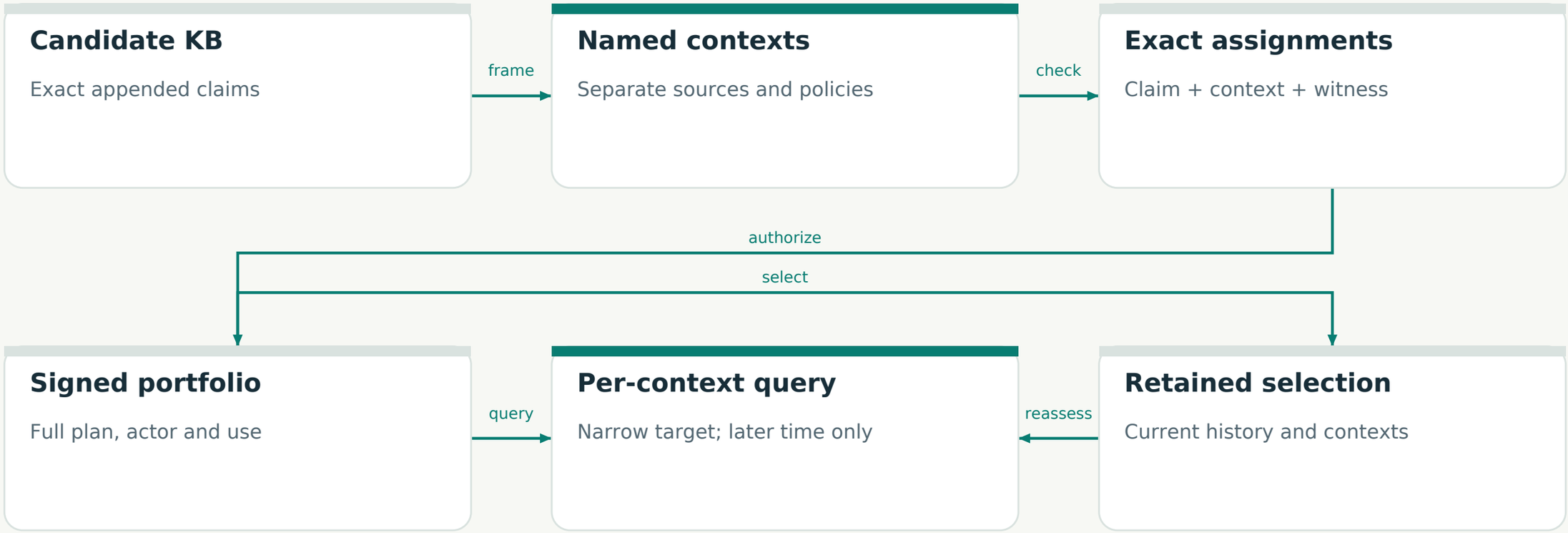
- Pure production kernel + Verus contracts
- Three-host reference harness and traceability
- Lean/Verus/conformance/mutation receipts

EXIT GATE

- All reference operations agree on complete responses; no Python runtime fallback.
- Selected actual Rust functions verify and are linked into the candidate binary.
- Unapproved assumptions/open obligations are zero; cross-prover correspondence limits are stated.

Verify named context portfolios

Authorize several contexts in one release without pooling their evidence.



BUILD

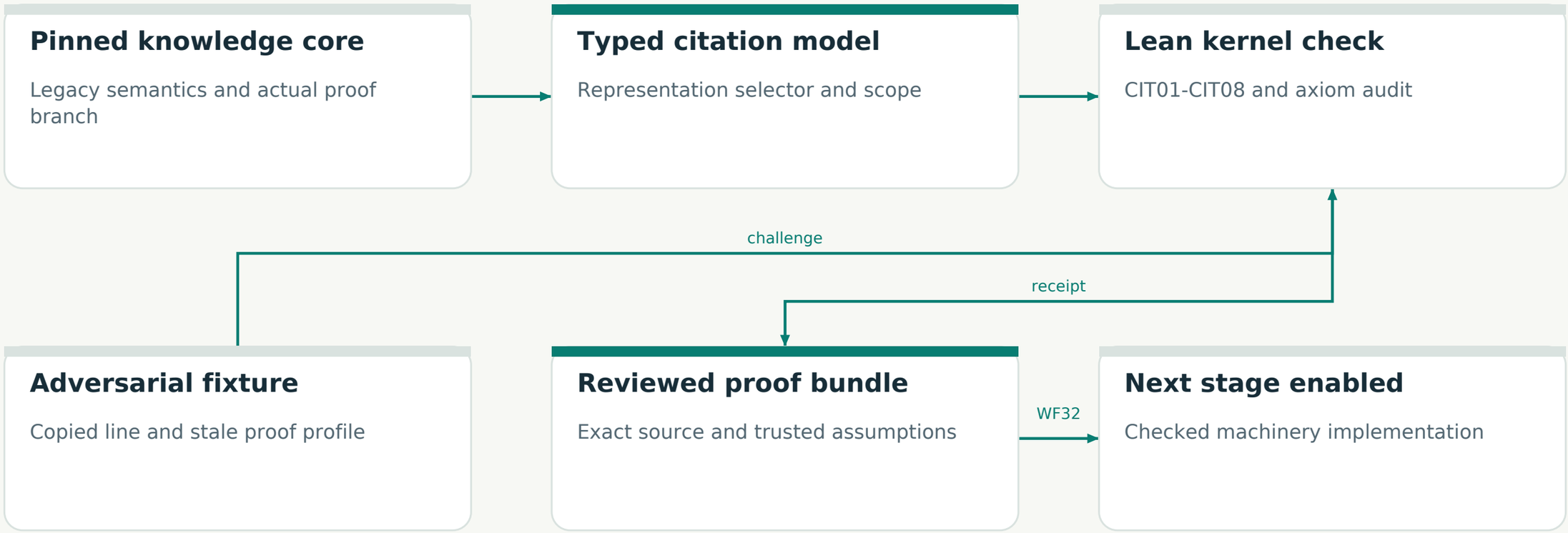
- kernel/portfolio + nominal records
- V12 and updated model/conformance receipts
- Context query, version and migration contracts

EXIT GATE

- Multiple targets remain separately assessed in one authenticated release.
- Cross-context borrowing, omitted assignments and implicit retirement fail.
- Historical selection preserves current corrections and context policy.

Prove citation binding and admission

Establish exact citation judgments and a verified admission floor before enabling new proposals.



BUILD

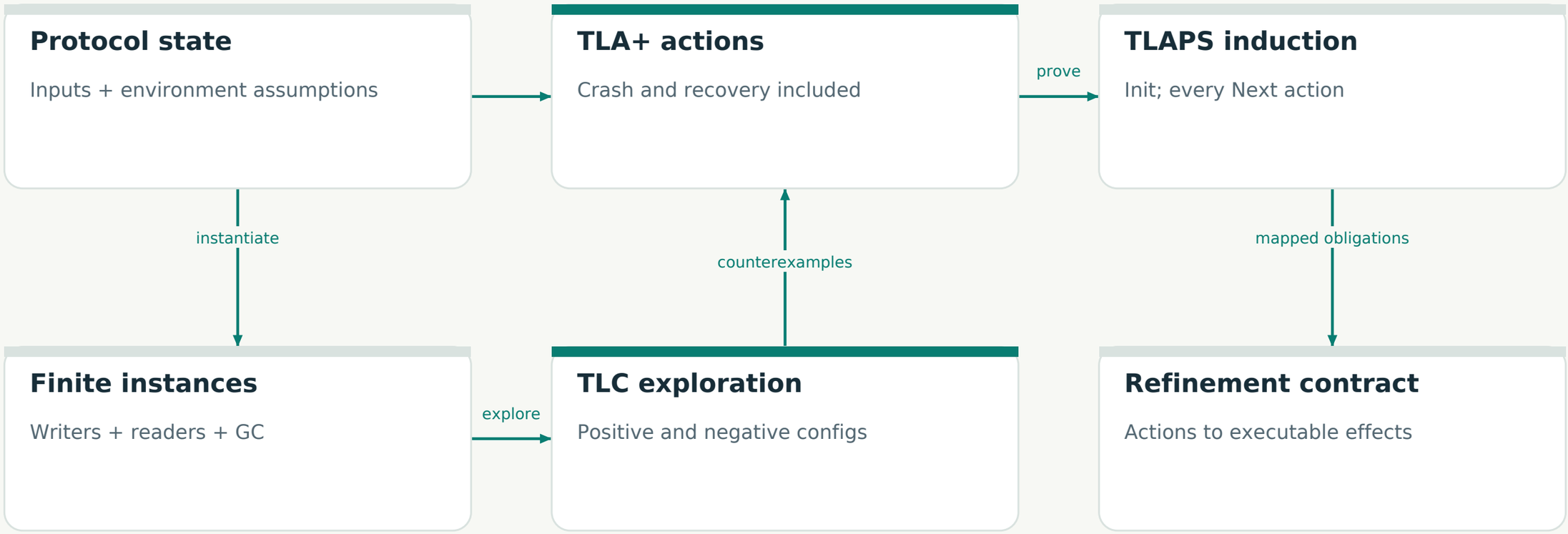
- Lean judgments and theorem map
- Checked proof bundles and assumptions
- Counterexamples and mutation results

EXIT GATE

- CIT01-CIT08 have audited receipts.
- Legacy interpretations are preserved.
- Missing or mismatched proofs block.

Prove the operational protocols

Find interleaving failures and prove the declared safety invariants before integrating effects.



BUILD

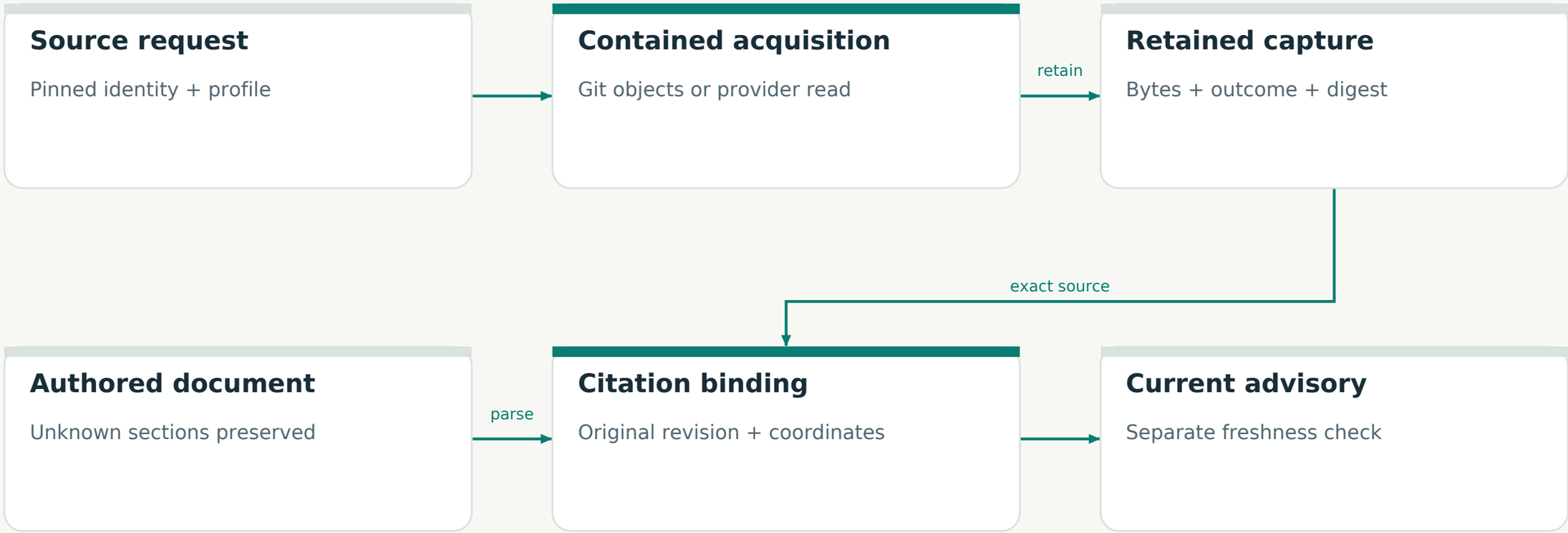
- spec/M01-M06, M09 and M11 + TLC/TLAPS modules
- Action/refinement/effect contract map
- TLC/TLAPS receipts + calibrated mutant registry

EXIT GATE

- Every named action and bridge has a checked safety obligation.
- All positive TLC runs finish; mutants violate the intended property, not parsing.
- No general temporal proof claim is inferred from the supported TLAPS fragment.

Exact sources, citations and filesystem effects

Acquire and retain exact evidence while preserving authored bytes and containment.



BUILD

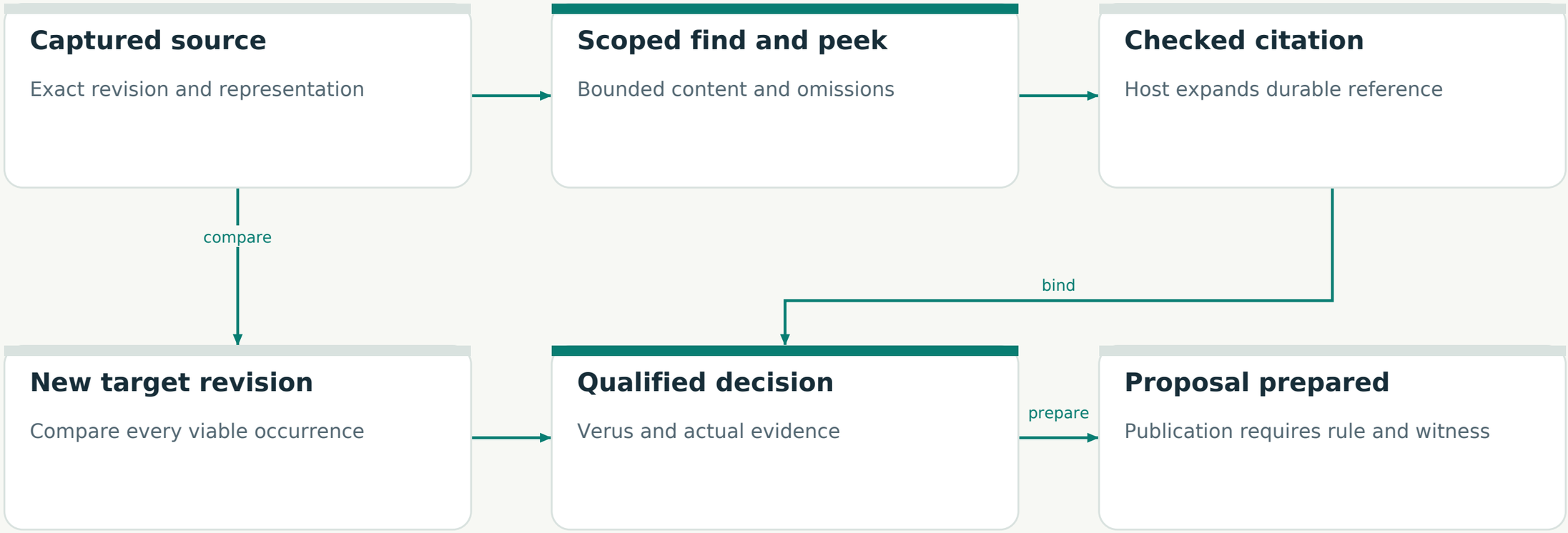
- sources crate + filesystem/Git adapters
- Lossless document and citation fixtures
- Real-source containment/publication receipts

EXIT GATE

- Historical citation bytes survive rename/refresh/correction.
- No lexical-path-only containment claim; symlink/TOCTOU profile is exercised.
- Missing/partial/denied sources cannot become complete empty evidence.

Qualify the compact citation reader

Bind the proven judgments to the shipping reader and proposal-admission checker.



BUILD

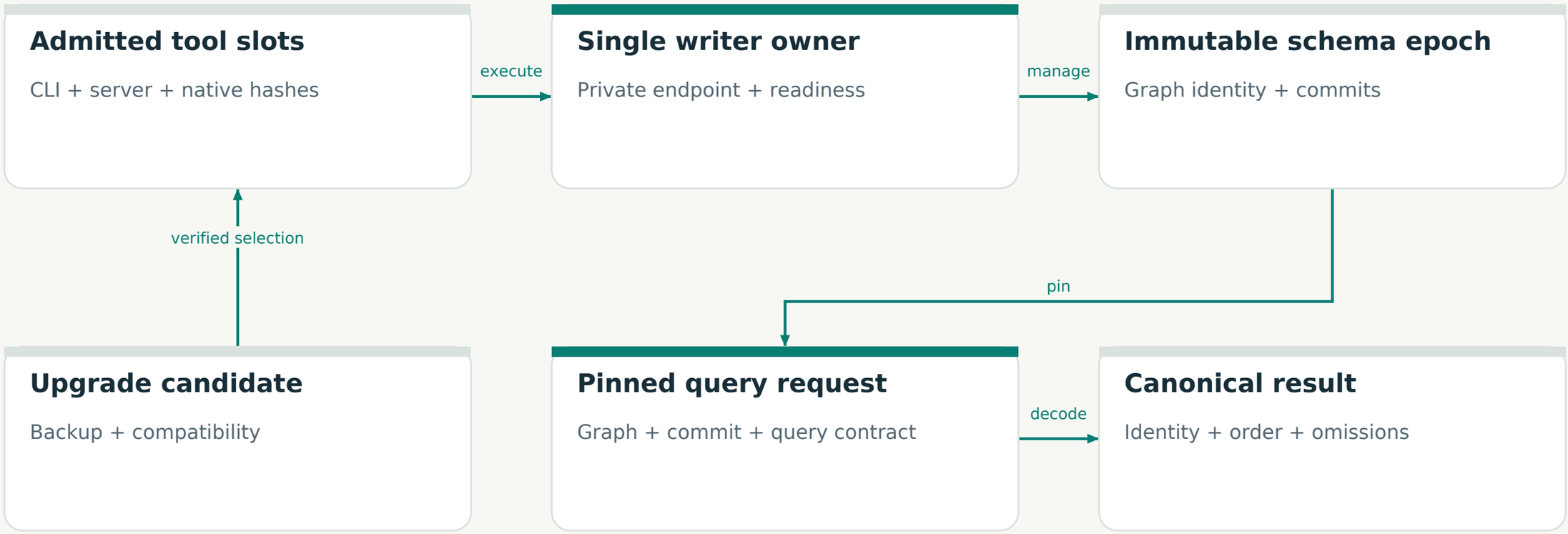
- Bounded reader and Rust checks
- Proof-bundle and witness validator
- Correctness and token evaluation

EXIT GATE

- No incorrect successor attachment.
- Compact results preserve coverage.
- All admission paths require proofs.

Managed Omnigraph and pinned queries

Provision one admitted native writer and bind every read to graph, commit, schema epoch and query contract.



BUILD

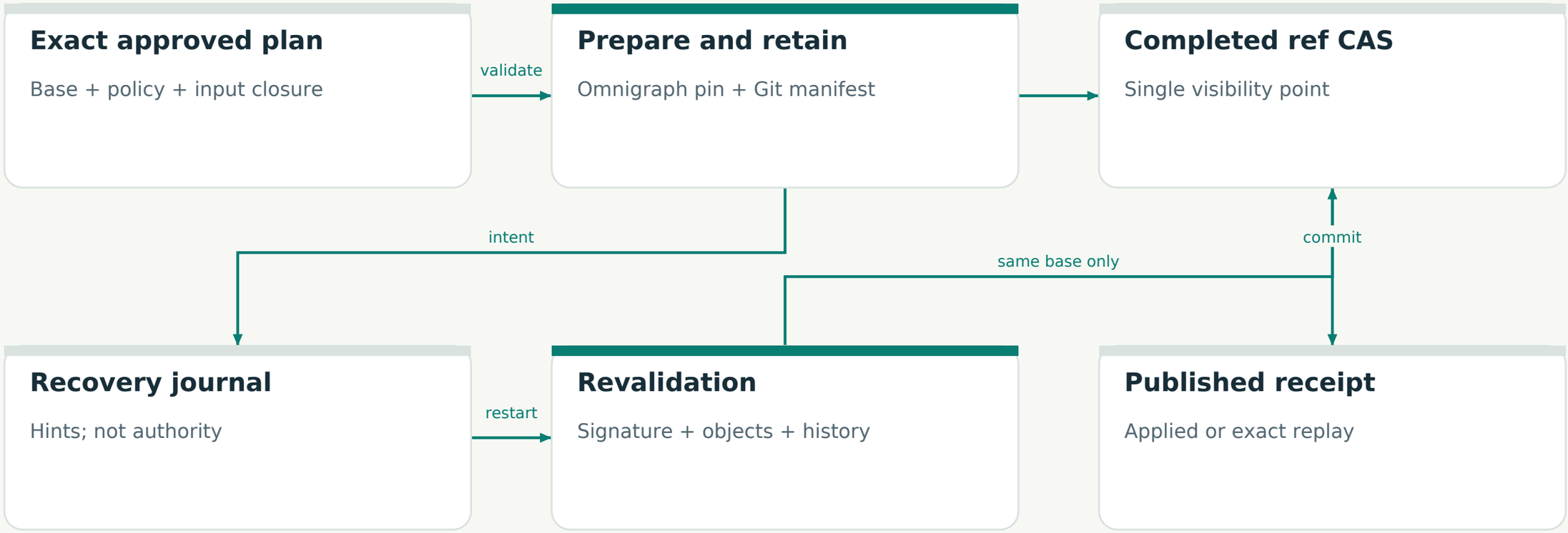
- omnigraph adapter + native supervisor/installer
- Epoch/query profiles + exact canonical mapping
- Integrated native lifecycle and pinned-read evidence

EXIT GATE

- Fresh/offline install uses admitted CLI/server and native dependencies.
- Queries cannot drift across graph, commit, epoch or query contract.
- Wrong pins, overlap, uncertain writes and unsafe rollback fail explicitly.

Authenticated publication and recovery

Publish a complete approved generation once and recover every interrupted outcome.



BUILD

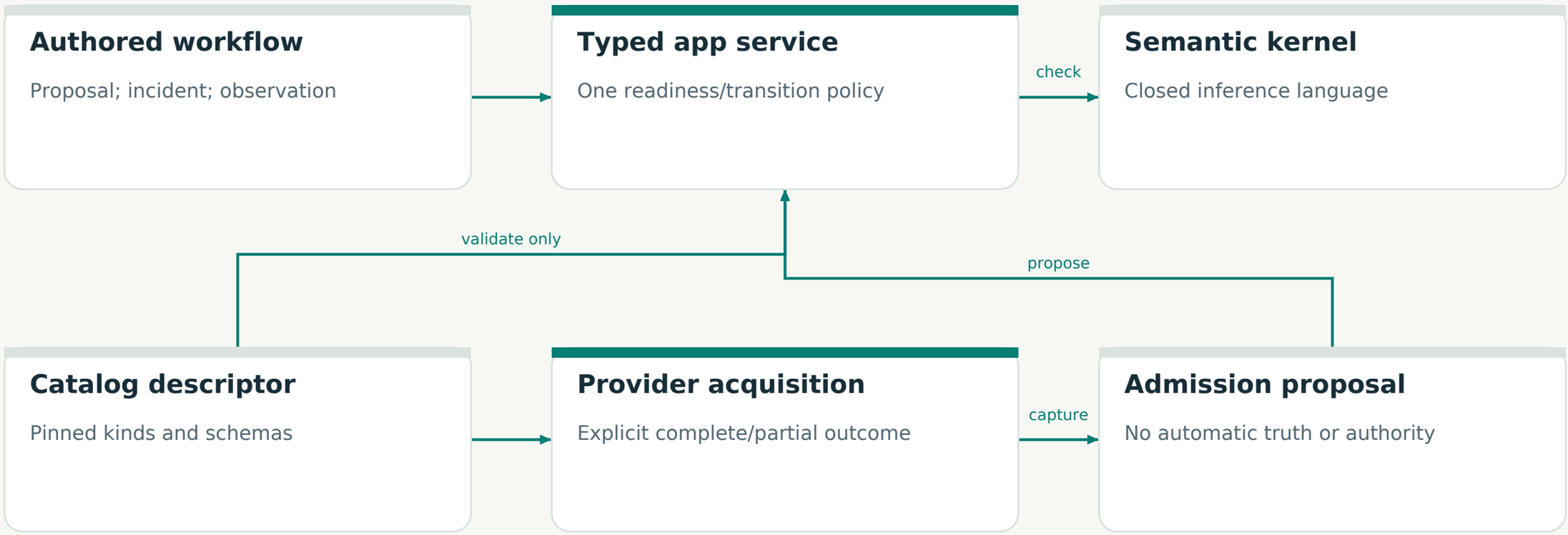
- Working source-to-approval-to-publication vertical slice
- Verified protocol reducer and action map
- Crash/race/replay receipts and operator recovery output

EXIT GATE

- One winner per expected predecessor; changed replay input conflicts.
- Post-CAS crash recovers as already applied without duplicate admission.
- Journal tampering/missing closure never creates publication authority.

Workflows and provider-neutral catalog

Preserve useful kctx workflows while keeping extensions outside the calculus.



BUILD

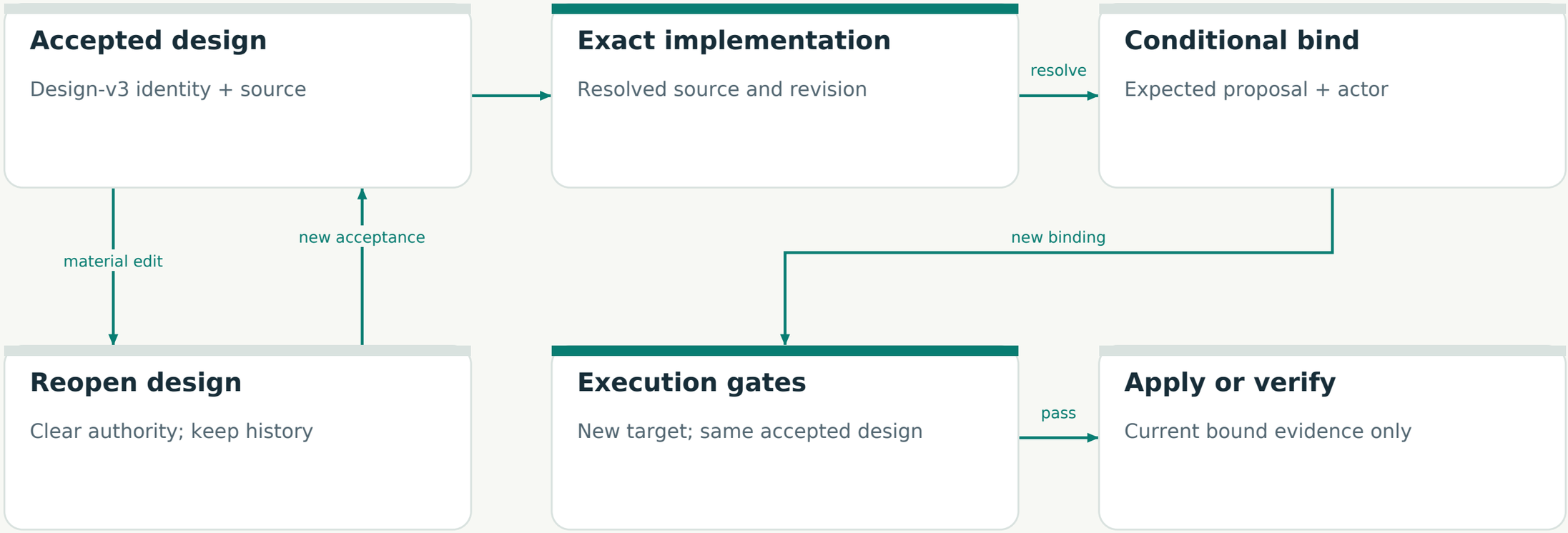
- Complete authored workflow services
- Catalog defaults + legacy provider profiles
- End-to-end multi-domain fixtures and updated operation map

EXIT GATE

- All baseline command behaviors have tested parity or approved explicit changes.
- Unknown extensions cannot silently fall back or inject inference/authority.
- Unknown authored content survives edits; gate readiness has one owner.

Bind implementations to accepted designs

Keep design approval stable while exact implementation evidence changes.



BUILD

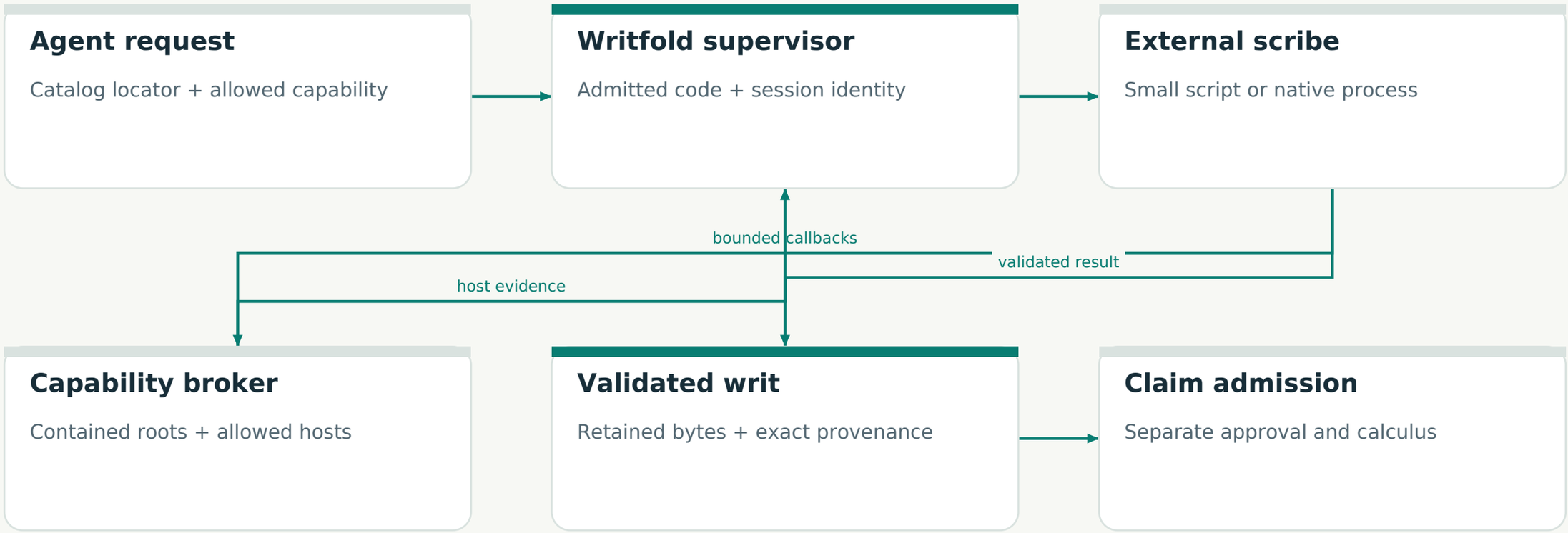
- kernel/design + authored projection/service
- V13 + M09 with caller/effect correspondence
- Migration, bind/reopen CLI and recovery fixtures

EXIT GATE

- Design can be accepted before implementation exists; bind resolves exact source/target.
- Rebind preserves approval but old-target passes cannot satisfy execution gates.
- Stale CAS, changed design and finalized-history rewriting are rejected.

Scribes and immutable writs

Let small scripts acquire evidence through a supervised, bounded and proved protocol.



BUILD

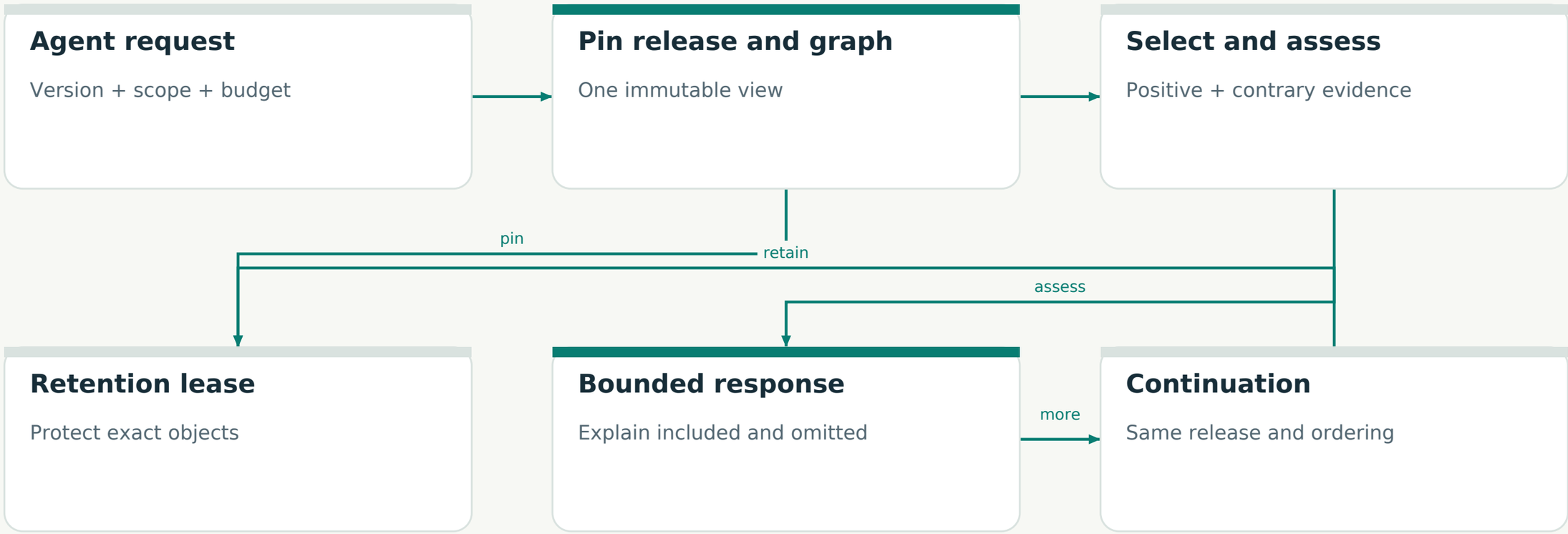
- writfold-scribes + catalog descriptors + optional script SDK
- M07/M08/V11 source-bound proof and calibration receipts
- Real local/HTTP integrations and script-author guide

EXIT GATE

- Only admitted code/runtime/grants can execute; no automatic plugin loading from sources.
- Late, duplicate, cancelled or old-session replies cannot create accepted writs.
- Complete acquisition requires validated stable transfer; no plugin can sign/admit claims.

Pinned context and agent interface

Deliver bounded usable context through one native CLI and stable stdio service.



BUILD

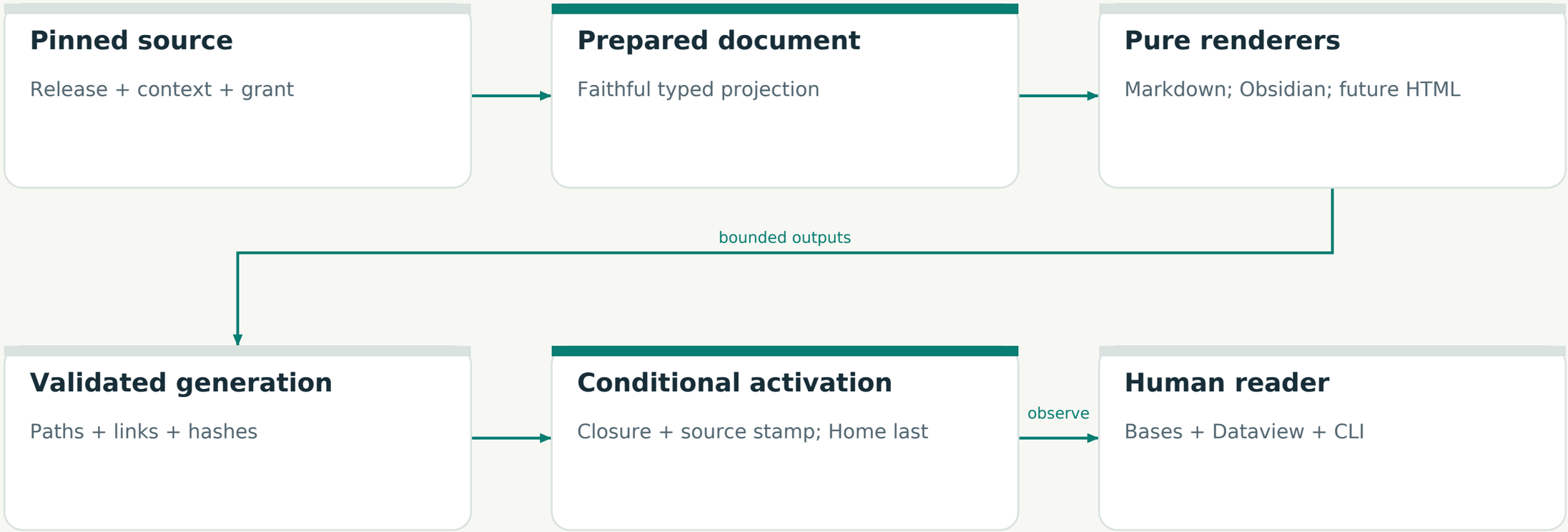
- Native CLI + initialized JSON-RPC service
- Context/explain/graph query and continuation types
- Installed transport and query completeness fixtures

EXIT GATE

- No command shells out to the Python host or another CLI frontend.
- Every page uses the same release, filters and ordering or refuses.
- Machine stdout is clean; omissions/contradictions remain explicit.

Readable views and Obsidian companion

Publish faithful, scoped knowledge views with readable Obsidian navigation and an optional CLI companion.



BUILD

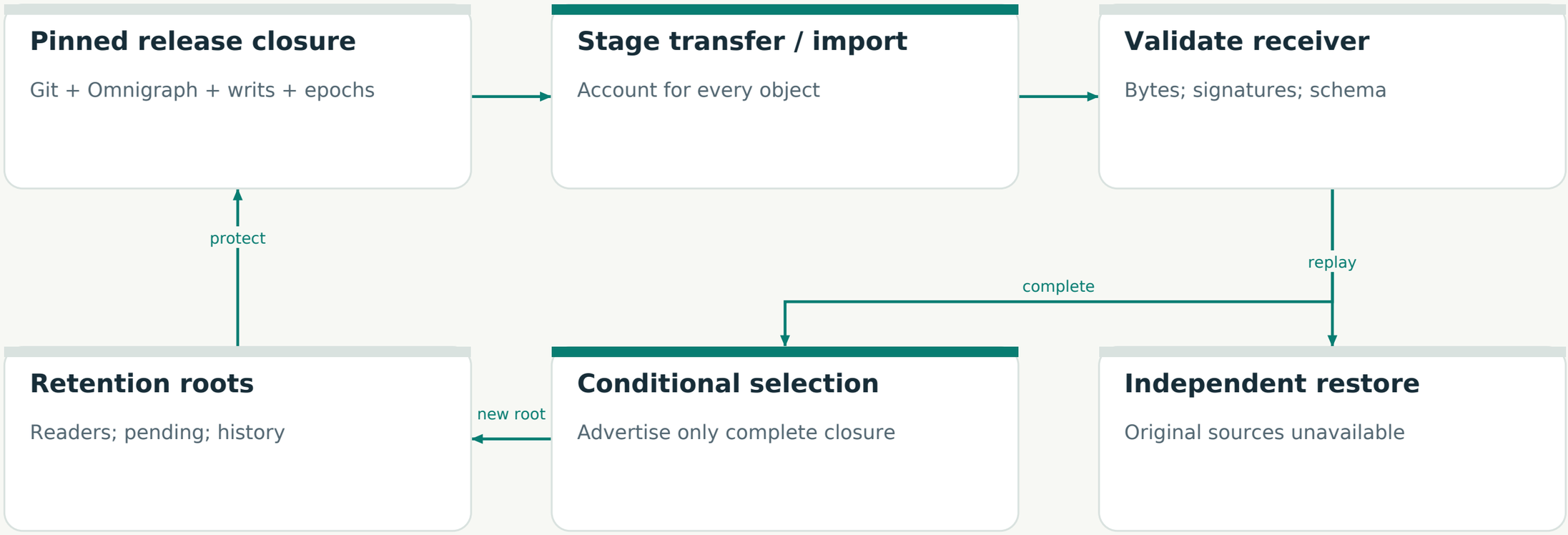
- Typed prepared documents + pure renderers
- Owned generation publisher and recovery model
- Native Obsidian profile and reading acceptance

EXIT GATE

- V14/M10 and all view isolation/recovery scenarios pass.
- Human files survive; stale views never masquerade as current.
- CLI/plugin profile and readable fallback pass on the admitted PC.

Migration, sync and retained history

Move and restore the full knowledge closure without losing authority or history.



BUILD

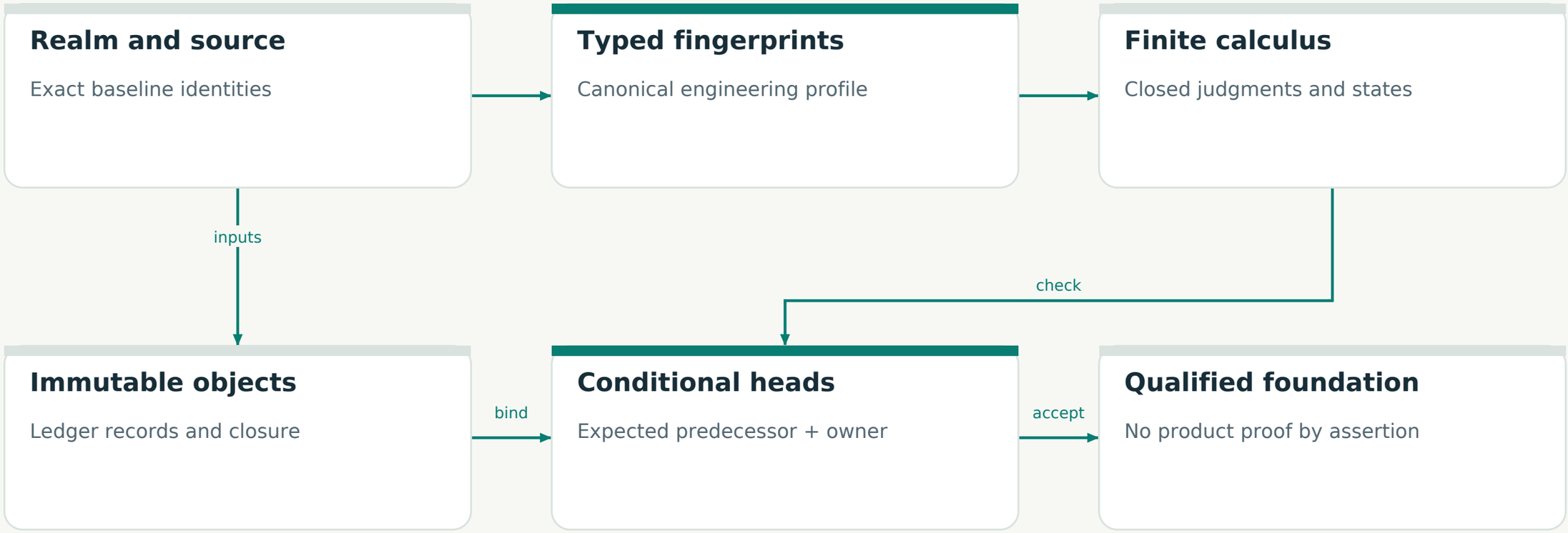
- Migration and independent restore tooling
- Local native/Git sync with closure receipts
- Retention/GC reducer, roots explain and race fixtures

EXIT GATE

- Every input has a disposition; repeats are idempotent; source-removal restore succeeds.
- Half-transfers never advertise incomplete releases; two clients do not erase history.
- Pinned readers and pending approved operations survive GC; rollback does not erase corrections.

Engineering identities and ledger

Establish exact identities, durable operational history and the conservative calculus boundary.



BUILD

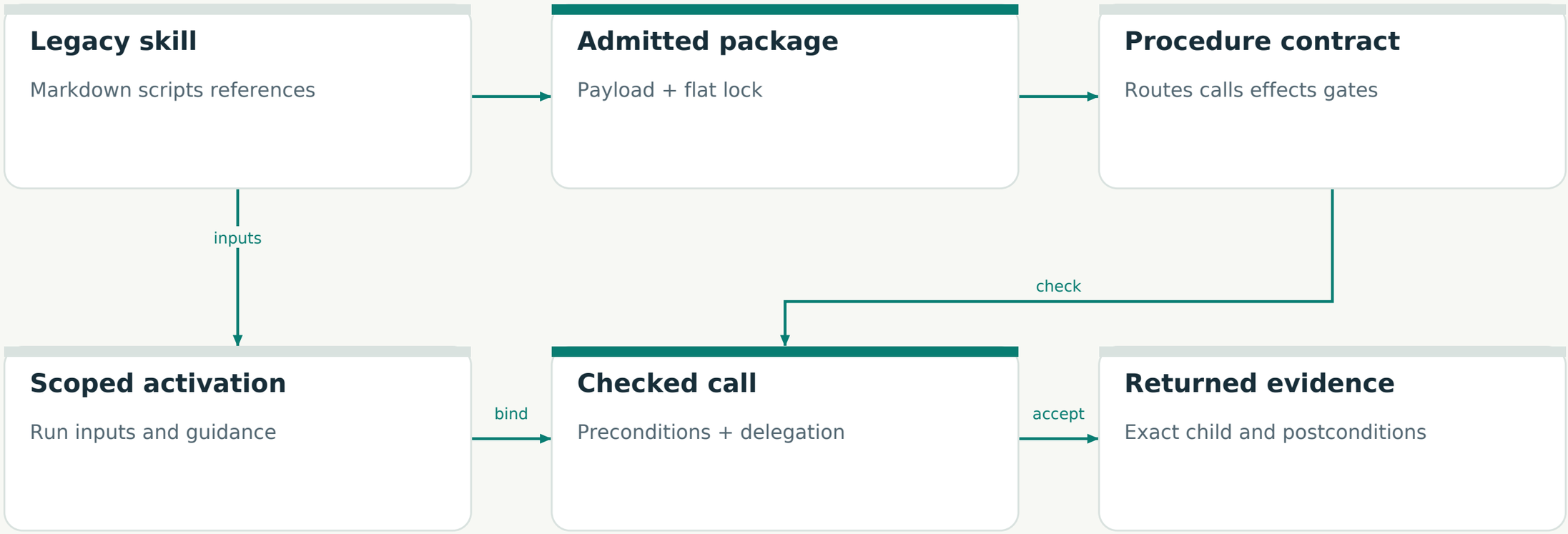
- Identity and codec contracts
- Durable ledger and head CAS
- Lean definitions and proof mapping

EXIT GATE

- No identity or authority substitution.
- Crashes preserve honest stream state.
- Golden codecs and model checks agree.

Skills and procedure composition

Preserve existing Markdown and scripts while checking loading, calls and effect ceilings.



BUILD

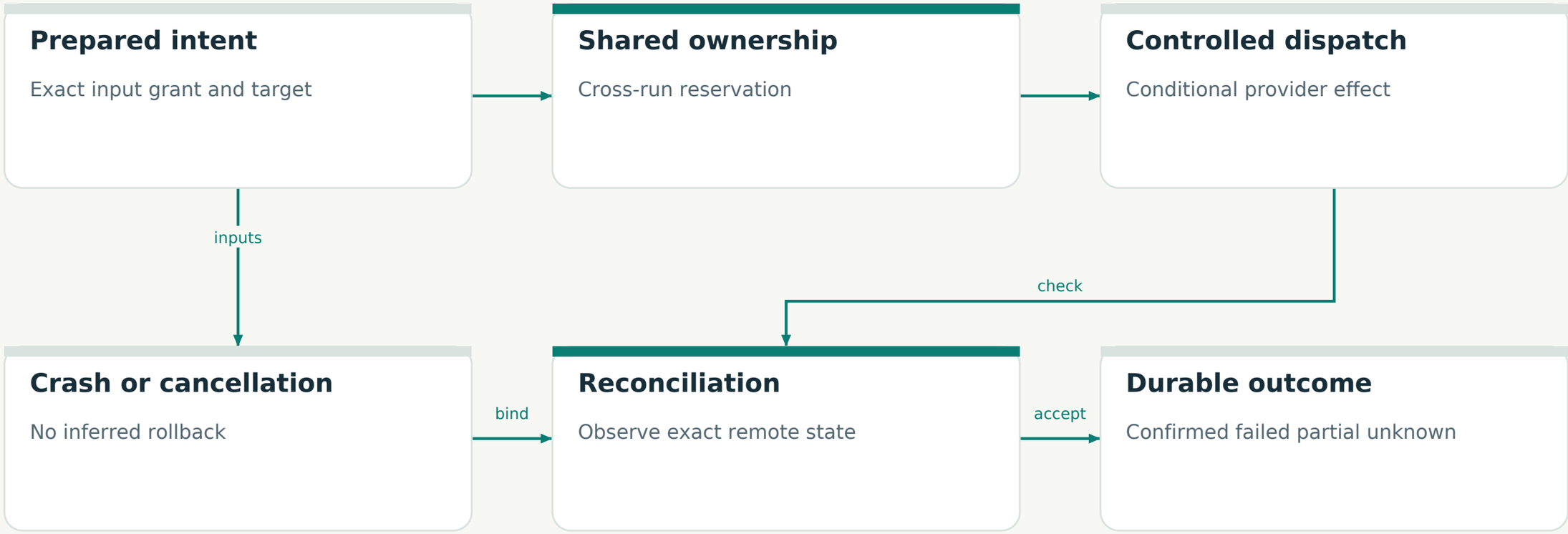
- Pinned skill import and loader
- Typed procedure contracts
- Nested-call qualification

EXIT GATE

- Prose cannot grant capabilities.
- Calls bind exact inputs and results.
- Active revisions never hot-swap.

Controlled effects and recovery

Authorize real effects through bounded adapters and shared resource ownership.



BUILD

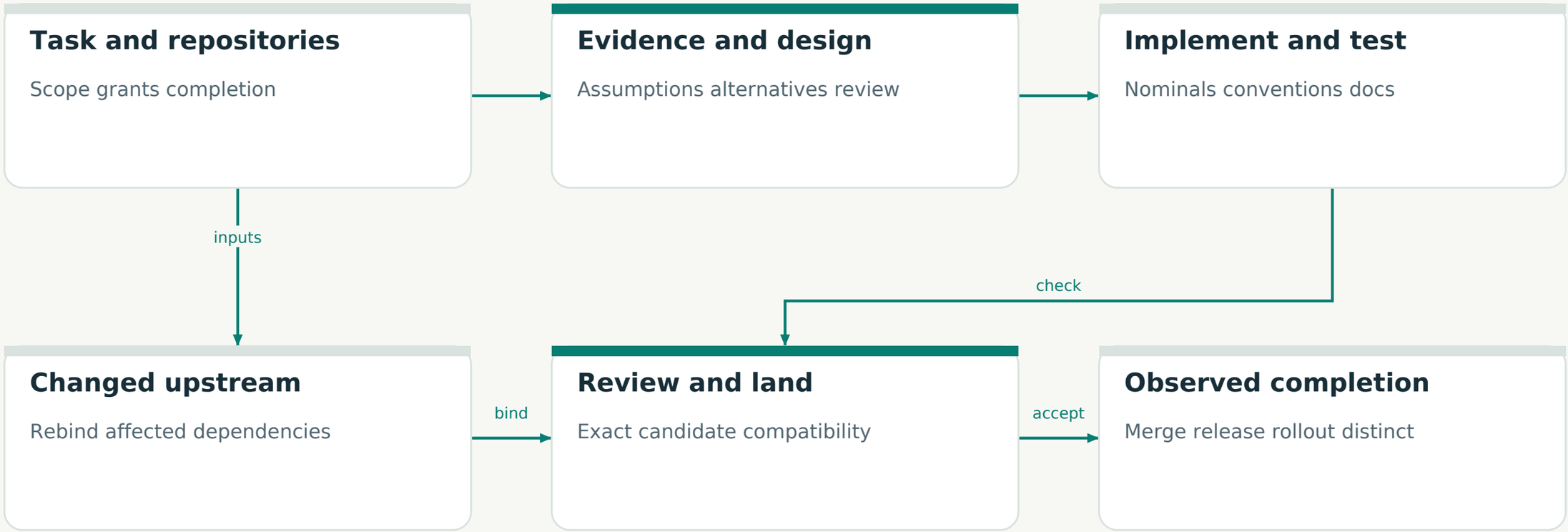
- Admitted effect adapters
- Shared ownership and fencing
- Crash and outcome reconciliation

EXIT GATE

- No stale or ungranted dispatch.
- Unknown outcomes remain explicit.
- Resources wait for real teardown.

Cross-repo author-change procedure

Deliver one complete change lifecycle across API, SDK and CLI repositories.



BUILD

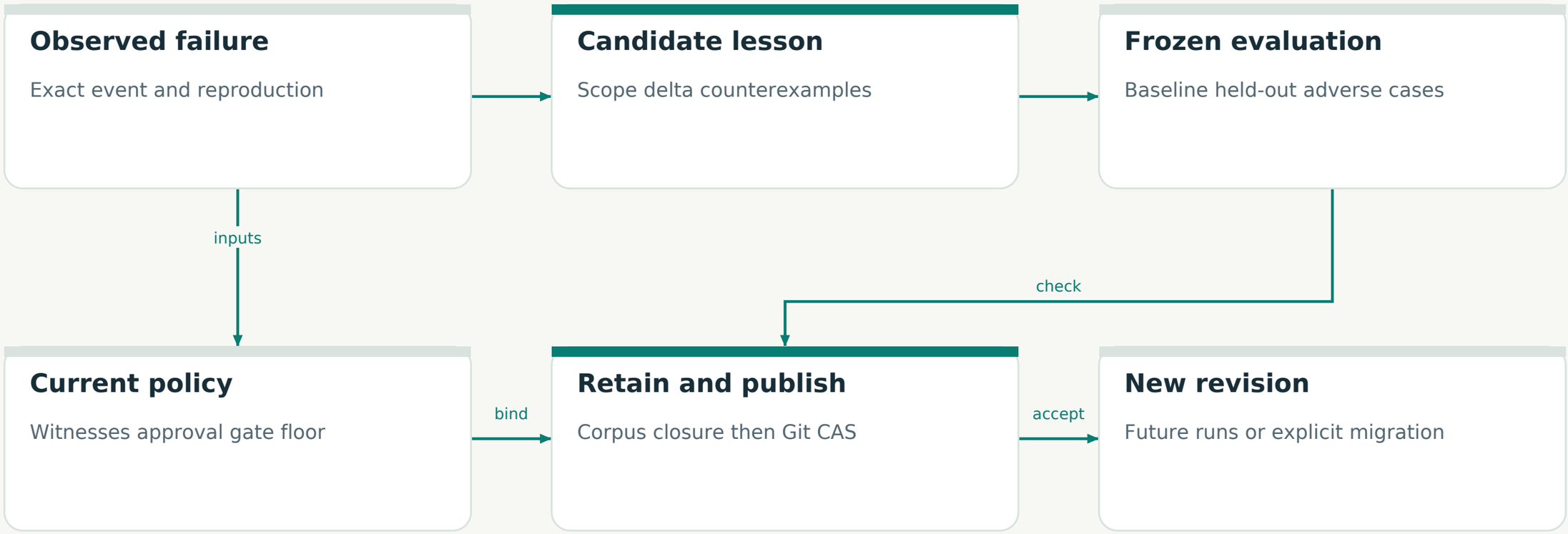
- Complete author-change procedure
- Cross-repo compatibility matrix
- PR and recovery evidence

EXIT GATE

- Tests bind the actual repo vector.
- Partial landing has a safe path.
- Review changes invalidate evidence.

Learning and reviewed promotion

Turn failures and feedback into evaluated revisions without self-authorization.



BUILD

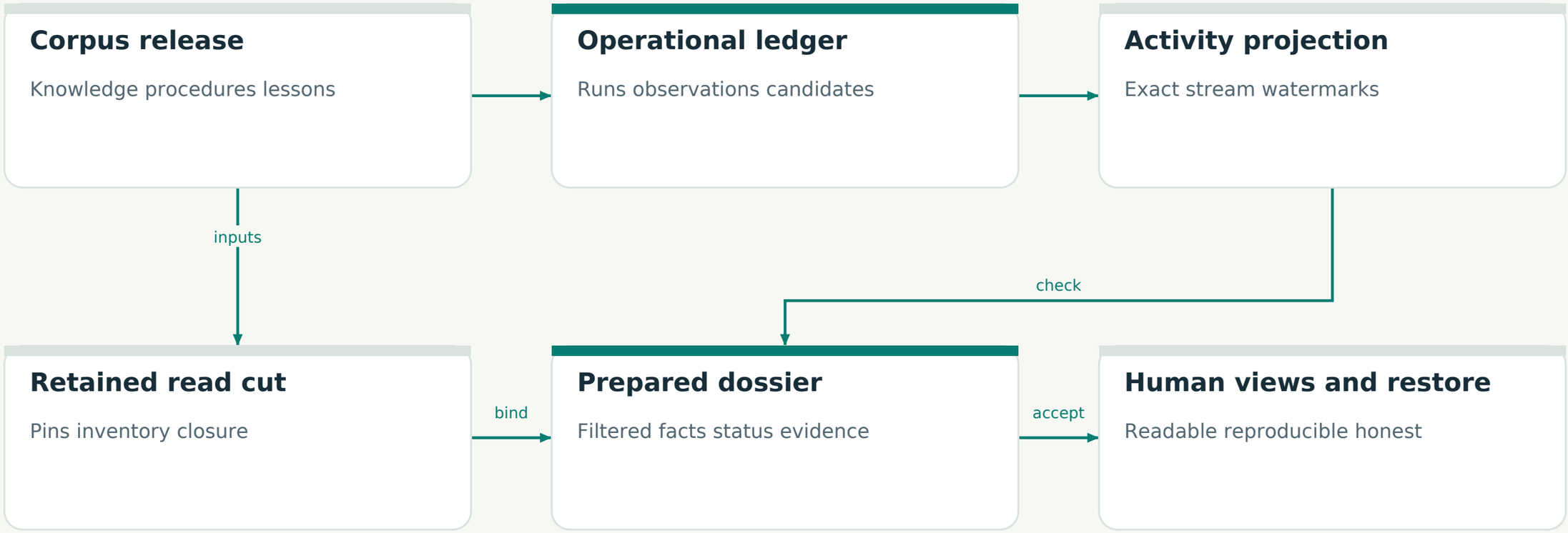
- Scoped candidate lifecycle
- Frozen empirical evaluation
- Authenticated promotion bridge

EXIT GATE

- Observations do not self-admit.
- Learning cannot weaken gate floors.
- Published evidence survives pruning.

Unified dossiers and restoration

Join all three semantic graphs through exact cuts and readable human views.



BUILD

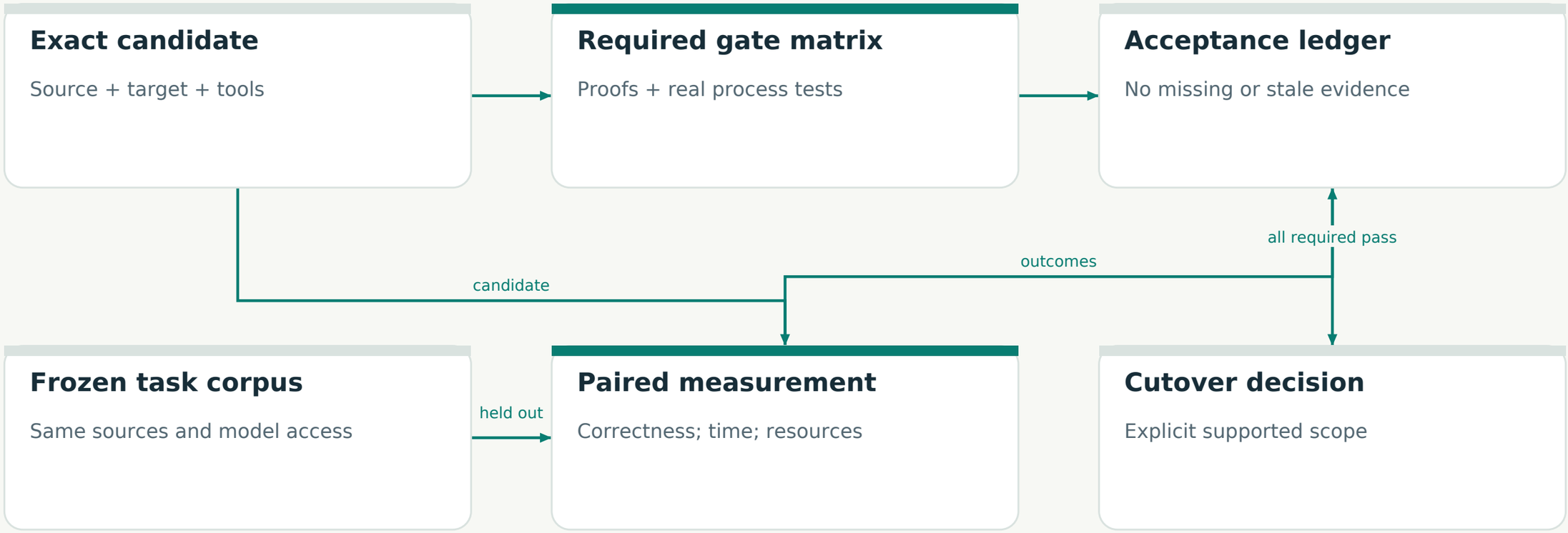
- Two-family storage and exact cuts
- Human change and learning dossiers
- Independent restore and rebuild

EXIT GATE

- Projection cannot authorize effects.
- Views disclose pins and omissions.
- Restore cannot replay old effects.

Qualify outcomes and Rust cutover

Demonstrate complete behavior, protocol evidence and measured usefulness on the candidate.



BUILD

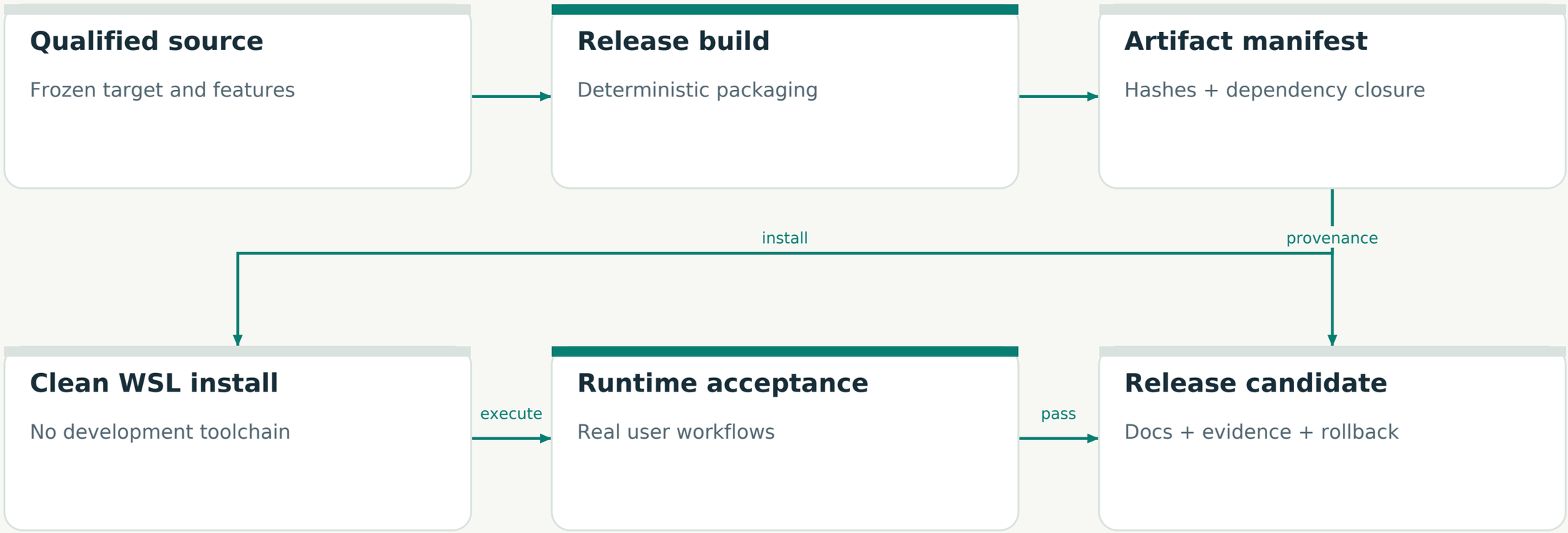
- Integrated qualification and traceability report
- Matched correctness/resource/outcome measurements
- Cutover decision + migrated caller contracts

EXIT GATE

- No unresolved deterministic candidate regression or required proof gap.
- Predeclared budgets/outcome policy met; limits and confounders disclosed.
- All delivered workflows run with the Rust artifact; no hidden Python fallback.

Package the native product

Ship a reproducible Linux/WSL artifact and an honest offline runtime closure.



BUILD

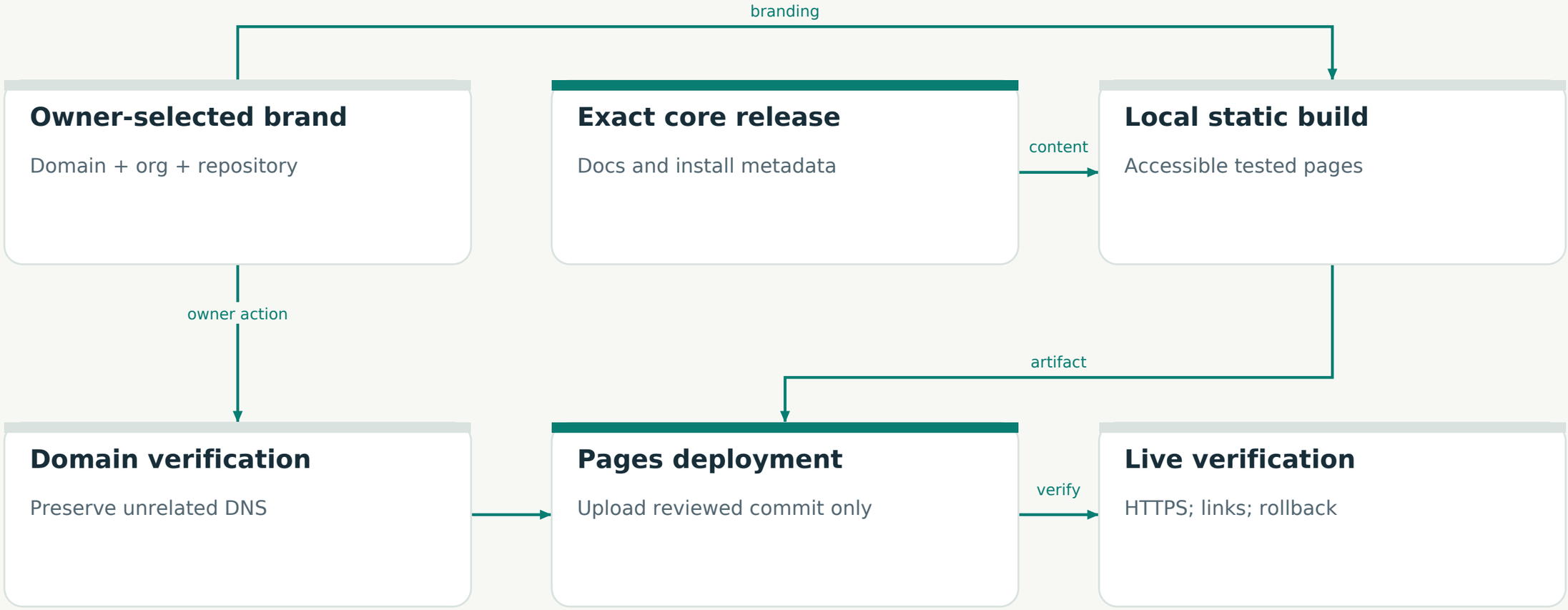
- Linux x86_64 release archives and offline bundle
- Checksums/SBOM/license/qualification closure
- Installed clean-machine acceptance report

EXIT GATE

- No hidden Docker/Python/proof runtime dependency; Git/Omnigraph closure is explicit.
- Bad download/update is refused; user data survives interrupted install/uninstall.
- Every release claim maps to exact artifact/source evidence.

Website and project launch

Present the real product through a polished static site tied to its exact release.



BUILD

- Static website source + rendered artifact
- Porkbun/Pages checklist and exact release links
- Site qualification and, if authorized, deployment receipt

EXIT GATE

- Install/examples match a real release; no exaggerated verification claim.
- Pages only deploys the reviewed static commit and never builds product code.
- Publication/DNS/purchase are reported distinctly from local preparation.

From a chat to a running change

A synthetic rehearsal on a configured WSL host. The first message comes directly after the durable start receipt.

TAREK

Track the pagination change across API, SDK and CLI through opening the reviewed PRs.

AGENT / AFTER RECEIPT

Run demo/RUN-017. Open it in a second WSL terminal. I’ll stop at opened, reviewed PRs.

```
writfold run watch demo/RUN-017
```

Live view

Gates opens on the current blocker. Press 2 for Pipeline, Enter for evidence and q to detach. Work continues in the driver session.

One snapshot

Use writfold run show demo/RUN-017. Add --at 34 to return to the exact event someone reviewed.

Back next session

Ask for the run ID again. The agent reads the saved binding and hands you the same command.

The ref names the same run from any worktree in the configured realm. Access is checked separately.

One driver for a logical change

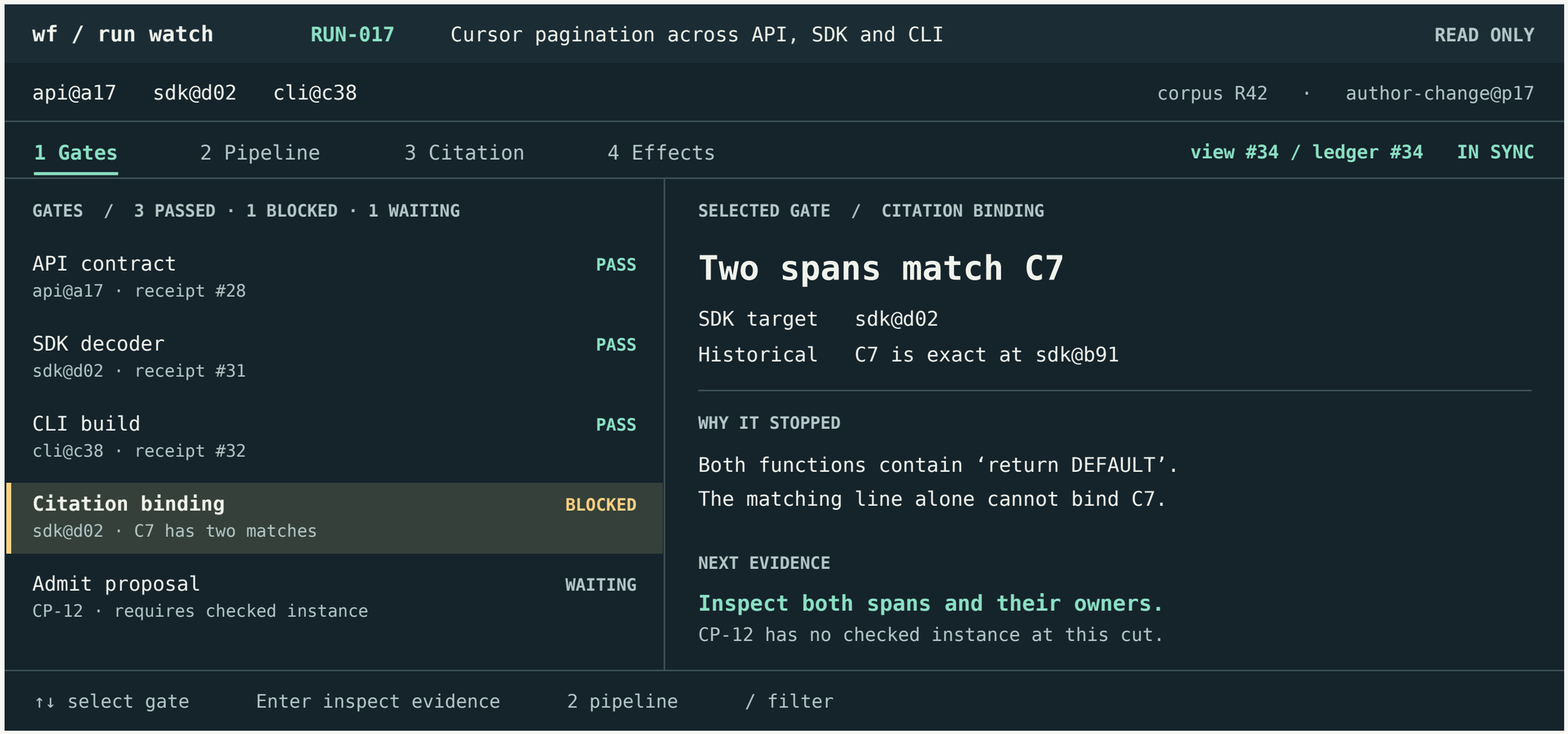
A chat can contain several tasks. Worktrees, reconnects and new chats can all belong to the same run.

01	**Asked for an ID**	Read the bound run. If two candidates fit, ask which one. If nothing is tracked, say so.
02	**Starting a tracked change**	Persist the task key and intake. Return the coordinator’s committed ref immediately. Recover the same key after a lost reply.
03	**Taking over a session**	Reload the pinned procedure and exact run. Check repo revisions, owner generation and unresolved effects before work resumes.
04	**Reporting progress**	Give the cut, the observed blocker or finish line, and the next evidence. A watch command opens a reader.

Implement the text snapshot and receipt first. Turn on the session driver and native view only after the relevant proof, runtime and protocol gates pass.

Gate inspector

Event 34. Three revision-bound checks pass. Two matching citation spans block CP-12.

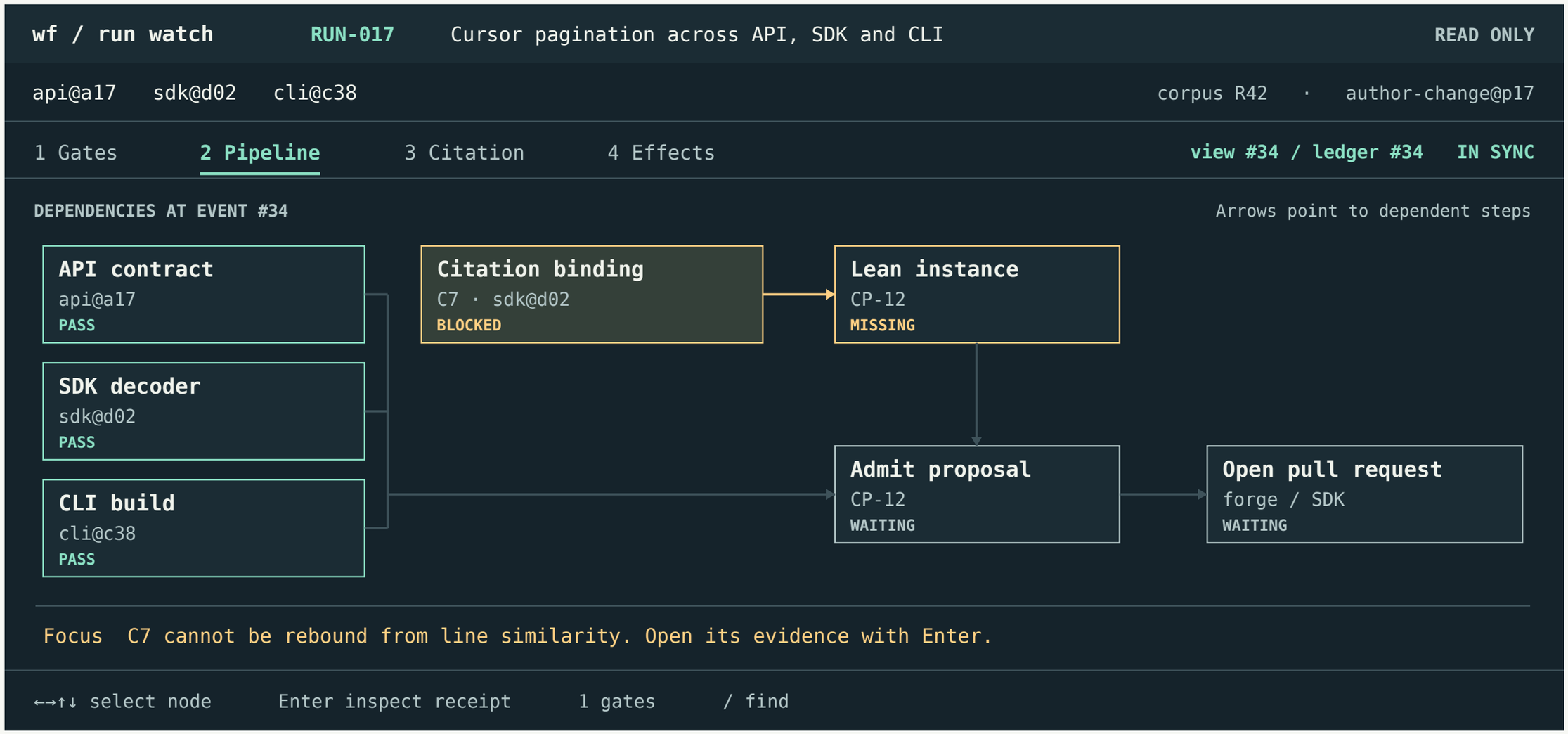


DEFAULT VIEW

A returning human should be able to name the blocker, the affected revision and the next evidence from this screen.

Pipeline trace

The same event as the previous page. Follow the citation branch to see what it holds up.



PRESS 2

Dependency arrows answer a different question: which later operation is waiting for this gate?

Citation inspector

Open C7 from either view. Both functions contain the same line at the target revision.

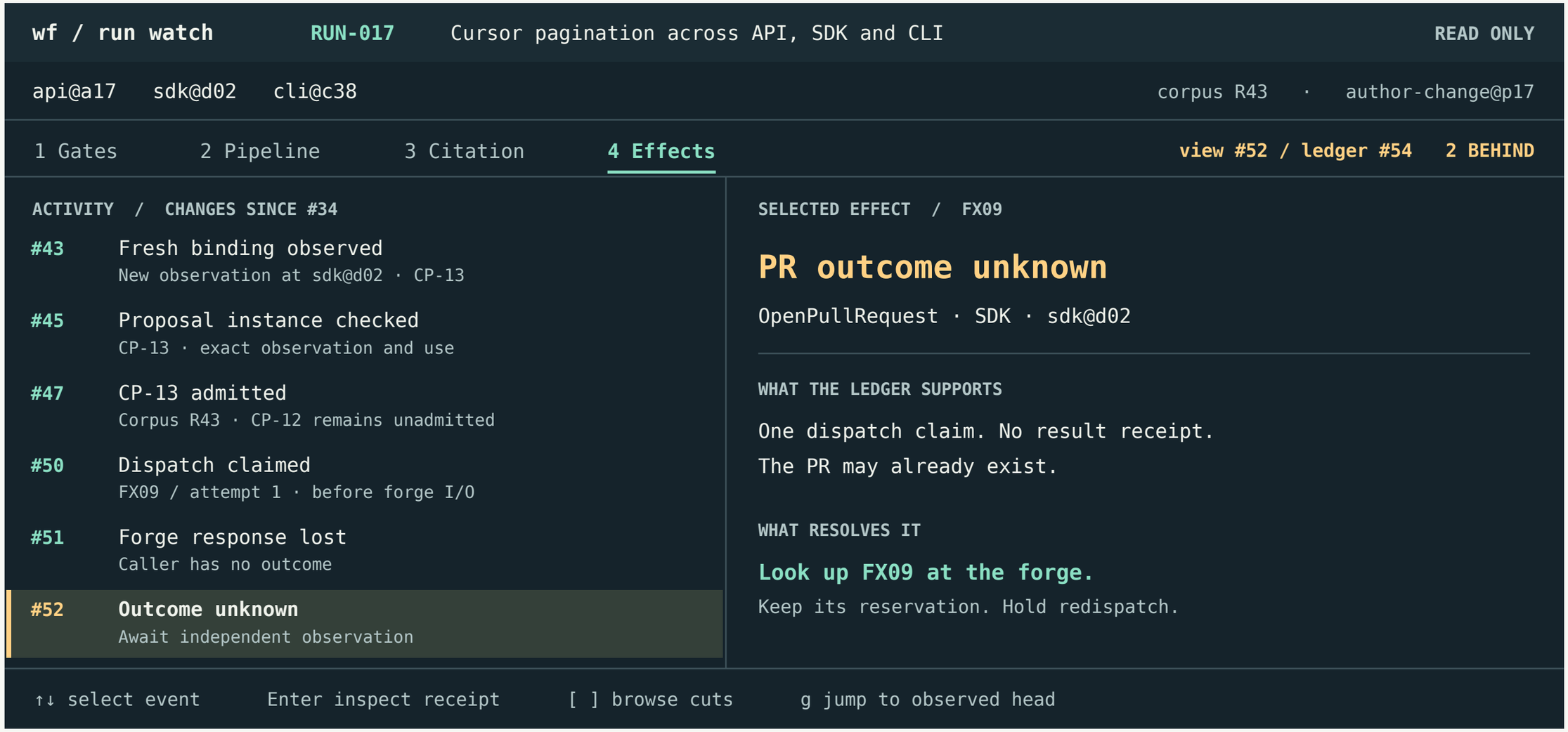
```
wf / run watch        RUN-017    Cursor pagination across API, SDK and CLI                READ ONLY
api@a17   sdk@d02   cli@c38                                  corpus R42  ·  author-change@p17
1 Gates      2 Pipeline      3 Citation      4 Effects                  view #34 / ledger #34   IN SYNC
C7  /  sdk / decoder.py                                  CP-12  ·  requested use: current SDK binding
CITED REVISION  /  sdk@b91                  TARGET REVISION  /  sdk@d02
EXACT AT ORIGINAL REVISION                  AMBIGUOUS  ·  2 TEXT MATCHES
1  def intended():                          1  def other():
2      return DEFAULT                       2      return DEFAULT
                                            4  def intended():
                                            5      return DEFAULT
CP-12 stays blocked. A fresh observation can support a separate proposal.
For a successor binding, supply an independent correspondence witness.
↑↓ inspect match      Tab change pane      Esc back to gate
```

PRESS ENTER

The original citation stays exact at its own revision. A current binding needs independent evidence.

When the reply goes missing

At event 52 the effect is unknown. Two newer ledger events lie outside the displayed cut.



TWO EVENTS BEHIND

An interrupted caller cannot conclude the PR failed. The effect keeps its identity while a reader reconnects.

Confirming the PR by lookup

At event 55 an independent forge observation confirms PR #187. Review and merge remain open.

```
wf / run watch        RUN-017    Cursor pagination across API, SDK and CLI                READ ONLY

api@a17   sdk@d02   cli@c38                                  corpus R43   ·   author-change@p17

1 Gates      2 Pipeline      3 Citation      4 Effects                  view #55 / ledger #55   IN SYNC

ACTIVITY  /  EFFECT FX09                          EFFECT RECEIPT  /  FX09

#50    Dispatch claimed                           PR #187 exists
       FX09 / attempt 1 · before forge I/O

#51    Forge response lost                        Confirmed     OpenPullRequest
       Original response unavailable              Target        sdk@d02
                                                  Identity      FX09 / attempt 1
#52    Outcome unknown                            Observation   #54 · independent lookup
       Reservation held
                                                  STILL OPEN
#54    Forge lookup observed                      PR review and merge
       SDK PR #187 carries effect marker FX09     No receipts at this cut.

#55    Effect confirmed
       PR opened · one dispatch claim

↑↓ select event     Enter inspect receipt     [ ] browse cuts     Esc back
```

CONFIRMED EFFECT

The receipt answers one question: did this PR open? The rest of the run has its own completion boundary.

Find the earlier incident

Synthetic example. Older SDK clients fail after an API rollout. No ticket number or exact error string is known.

AGENT ASKS

“Have older callers failed when a service added a response field?”

Scope: checkout / corpus R42 / budget: three case handles

01 FIND

Unknown-field exception

A historical SDK case mentions a strict decoder. A retry-loop report also matched the endpoint.

02 PEEK

Read the discriminator

Open the retained evidence. In the old incident the decoder threw before the next-page request.

03 TEST HERE

Check today’s failing client

Look for the exception at the affected version. If requests succeed and tokens repeat, investigate the retry loop instead.

The historical fix remains a hypothesis. Cite the old evidence and the new observation separately.

What a match can establish

The ranker may be approximate, incomplete or hostile. A handle still has to resolve inside the pinned, authorized context.

PROVED	**Search leaves authority unchanged**	An independent checker verified trace erasure by induction for arbitrarily long finite traces, plus five guard laws. Deliberately broken proof cases failed.
EXPLORED	**276 states and 1,640 edges**	The finite Python model found zero violations in the guarded variant. Removing each of three checks produced a counterexample.
PENDING	**Lean and protocol gates**	The packet contains Lean proof terms and a TLA+ model. Lean kernel, TLC, TLAPS and Rust refinement receipts remain open.
MEASURE	**Do the right cases surface?**	Compare a skilled grep workflow with lexical and hybrid retrieval. Count tokens through the final defensible citation, including failed investigations.

An empty result can coexist with a real incident. A visible result can still have a different cause.

Where the graph earns its place

Review of the 0.14.0 source pin. Promote a capability only when it answers a concrete Writfold question.

PILOT	**Search inside relationships**	Combine lexical and vector ranking with dependency traversal. A correlated evidence query can distinguish an observed symptom from a passing mention.
USE	**Named, bounded queries**	Bind one reviewed query and scope to find. Keep graph syntax, limits, revision selection and admission authority in the engine.
QUALIFY	**Explain plans and snapshots**	Inspect query plans for broad scans. Replay the pinned source and retained hit list. Consider native diff later if it reduces rebuild cost.
DEFER	**More agent write surfaces**	Branches per agent, generic mutating MCP and a separate vector database add no direct value to the first incident workflow.

Enable hybrid only if it improves top-three recall and lowers tokens per correct investigation without extra unsafe transfers.

Search the whole knowledge base

One find call can bring back several kinds of evidence. Each hit keeps its own revision, source and role.

ONE QUESTION

“How have we safely changed a response schema across repos?”

DESIGN	**A coexistence decision**	Open the accepted decision and check whether anything superseded it.
REVIEW + CODE	**A PR comment and a test**	Open the diff and its disposition. Try the regression against the current decoder.
PROCEDURE	**An author-change skill**	Inspect the pinned procedure and preconditions. Execution requires its own grant.
INCIDENT	**A previous client failure**	Find the historical discriminator and check it in today’s telemetry.

Incidents are the first benchmark. Service dossiers and admitted lessons also belong in discovery.

What did the proof checker do?

Metamath lets you write a claim as a sequence of explicit steps. A small verifier checks each step against the rules.

Two histories in the abstract model

A	Read design → read review → submit checked proposal P
B	Submit the same checked proposal P

Why the outcomes agree

Read steps preserve admitted state. For the same starting state and the same sequence of checked submissions, induction covers any finite number of intervening reads.

What the agent can still change

Reading the review may persuade the agent to submit a different proposal. That changes the inputs to the comparison. No theorem here says a hit is relevant or that its historical fix applies today.

Metamath supplied an independent check. The project’s Lean and Rust qualification gates remain open.